

数値計算の手引き

志村昂輝

2026 年 3 月 20 日

一級品として表れる諸々の技術や芸当は、その自由自在さで我々を驚かせますが、その背後には素人の知りえない膨大なルールと型があります。研究も例外ではなく、よりよい研究で発揮される個性はまずある一定の型やルールを身に着けた後に生まれています。研究における型を身に着ける最初的手段は、例えば学部生のうちには標準的な教科書を読んで知識を増やすことでしょうが、これは全くもって序の口にすぎず、一人前となるためには配属された研究室で師弟関係を結び、その研究室が売りとするスタイルを吸収していかなければなりません。

問題となるのが、この研究室の売りは一朝一夕に掴めるものではないということです。まず師弟関係の下で、弟子として師匠の聲咳に接しているかどうか、素人とそうでない人とを分ける最初の分水嶺となります。将来自分の同業者になると見込んだ人間に対して、師匠はその分野の共通言語や常識を叩き込みますが、一人前とみなされるにはその量と厳しさをもってしても数年を要します。修士と博士合わせて5年かかるのは、この修行が並大抵のものではなく、素人から業界人へと決定的に変化させる重大なプロセスであることを示唆しているでしょう。

本稿は以上2つの困難を克服するために作成されています。つまり、加藤研内部で共有されている重要だがインアクセシブルな技術をまとめ、研究を始める際に典型的に出くわす困難とその対処法を示すこと、及び内容を極力端的に整理し、一学生から研究者になるための経路を効率よく整備することです。ただし、私の特殊なバックグラウンドに起因する限界について読者にお断りしなくてはなりません。まず加藤雄介研究室で扱われるテーマと技術は広く、私がここで示す内容はそのほんの一部にすぎないことです。もとより本研究室の強みは数値計算よりも微分方程式の求解や特殊関数の操作といった解析計算に存在しており、本稿の内容はむしろ傍流といえるかもしれません。主流といえるそちらの技術については、読者の皆様が教員との長い議論や共同研究の中で少しずつ身に着けたり、あるいは今後誰かが同様にドキュメント化してくれることを期待いたします。

第1章では、数値計算や解析の作業に適したプログラミング言語の紹介と簡単な文法の解説、及び環境構築の仕方について述べます。また第2章ではC++に限定して代表的なツールとその使い方について紹介します。

強相関物質の研究スタイルにはかなり決まった型が存在していますが、出発点として死活的に重要なのがハミルトニアン of 構築とその対角化です。第3章では、強束縛模型について軽く説明した後、ハミルトニアンを実装する方法、及びその対角化の方法について述べます。ハミルトニアン of 対角化により得られた固有値がエネルギー分散ですが、実際に研究したり論文を書いたりするうえで必要なバンド図やフェルミ面の書き方についてもここで解説します。

最後に、本書はあくまでも研究室内ドキュメントであり必要に応じて参照すればよいと思いますが、内容をしっかり理解したい方は掲載されているプログラムを自分の手で実装してみるとよいと思います。最初はコードの写経でもよいですが、書き換えや順序の入れ替えで挙動がどうなるかをチェックするとより理解が深まるかと思います。

Chapter 0

目次

第 1 章 プログラミング言語の選択と環境構築	5
1.1 数値計算向きの言語 (C++, Fortran)	5
1.1.1 C++について	5
1.1.2 Fortran について	6
1.1.3 Python3 について	6
1.1.4 近年注目されている言語 (Julia)	7
1.2 C++で使える高速数値計算ライブラリ	7
1.2.1 BLAS/LAPACK(線形代数用ライブラリ)	7
1.2.2 FFTW3(フーリエ変換)	8
1.2.3 OpenMP(並列計算)	8
1.3 計算環境の準備	8
1.3.1 WSL の有効化	9
1.3.2 Visual Studio Code の導入	10
1.3.3 ファイル操作のコマンド	13
1.3.4 シェルスクリプト	14
1.3.5 Makefile	15
1.4 GitHub の導入 (やや発展)	18
1.4.1 インストール	18
1.4.2 履歴の残し方	19
1.4.3 SSH 接続	19
1.4.4 リポジトリへの変更の反映	20
1.4.5 その他	21

1.5	C++ / Python 開発環境の構築	21
1.5.1	インストール	21
1.5.2	動作確認	22
第 2 章	C++による数値計算の基礎	23
2.1	コンパイルおよび Makefile の作成	23
2.2	C++による対角化	24
2.2.1	zheev の引数	25
2.2.2	コンパイルおよび実行	25
2.2.3	対角化の手順	26
2.3	C++による FFTW3 を用いたフーリエ変換	27
2.3.1	FFTW における型	27
2.3.2	plan とは何か	28
2.3.3	フーリエ変換の手順	30
2.4	C++による OpenMP を用いた並列計算	32
2.4.1	いかなるときに並列化が有効か	32
2.4.2	基本的な使い方	32
2.4.3	parallel / parallel for	33
2.4.4	OpenMP における変数の扱い (shared/private)	34
2.4.5	reduction	34
第 3 章	強束縛モデルの解析	36
3.1	2次元正方格子の強束縛モデルの構築	36
3.2	バンド図とフェルミ面	38
3.2.1	バンド図の描画	39
3.2.2	フェルミ面の描画	39
3.2.3	化学ポテンシャル μ の自己無撞着な決定	40
第 4 章	付録	41
4.0.1	LAPACK を用いた対角化サンプルプログラム	41
4.0.2	FFTW を用いた高速フーリエ変換サンプルプログラム	47
4.0.3	OpenMP による並列計算サンプルプログラム	51

Chapter 1

プログラミング言語の選択と環境構築

本章では数値計算に用いる言語にどのようなものがあるか、また代表的にどのような機能があるかを俯瞰した後、実際に環境構築および整備を進めていきます。すぐにでも環境構築をしたいという方は3節の「計算環境の準備」の項を読んでください。

1.1 数値計算向きの言語 (C++, Fortran)

数値計算という目的に絞ってもプログラミング言語の選択肢は多岐にわたりますが、物性物理で格子模型や多体問題を扱う場合、計算量は系の大きさや自由度の増加とともに急激に増大します。したがって、実行速度やメモリ配置を意識する必要があり、C++やFortranはそれにふさわしい選択肢の一つとして挙げられます。明示的に型が指定されるために、数値誤差のふるまいを把握しやすくなるのも重要です。

1.1.1 C++について

C++はC言語が基盤となっていますが、クラスやテンプレートといった機能を導入することで大規模なプログラムの構築や保守に向いています。本稿で主に用いる言語です。コンパイラもフリーで手に入り、特にLinuxでは標準装備されています。Windowsでもコンパイラがフリーで入手できるようです。

数値計算上一番大きなメリットはなんといっても、実行速度の速さです。C++はコンパイル型言語で、ループなどが機械語に近い形で最適化されるために、行列演算や反復計算で高い性能を発揮します。また配列の確保や解放を動的に行えば巨大配列を扱うこ

ともできるので、波数空間上の物理量の情報を格納する場合に非常に効率がよいです。数値計算のライブラリも充実しています。

デメリットとして、メモリ管理を意識した低水準な記述が前提となっているために、数値計算にバグが混じりやすくなることが挙げられます。Fortran に比べると覚える概念が多く、特にポインタは大部分の学習者が躓く場所として悪名高いもので、言語に対する正確な理解と注意深い実装が求められます。それでも私がここで C++ をお勧めするのは、学習コストが高い分ほかの言語を学習する際のハードルが下がるであろうことや、Fortran に比べるとできることが多いこと、最後に身も蓋もありませんが、筆者が Fortran よりも長く触れているためにドキュメントを作りやすいことがあります。例えば個人的に文字列操作は Fortran よりも C++ のほうが便利だと考えています。

1.1.2 Fortran について

数値計算の言語としては Fortran も依然重要な選択肢といえます。物性理論だけでなく、科学技術計算に特化した言語として長い歴史を持つために、研究室によっては保守性の観点から Fortran によるコーディングを強いる場合もあります。これはオリジナリティ保護の観点から理にかなっていて、研究室の強みが Fortran に支えられており、いわば研究室独自のライブラリとして使いまわすことができるのです。大規模プロジェクトならなおさら、既存の技術的な資産を一人で書き換えることも不可能でしょう。しかしながら加藤雄介研究室は数値計算の研究室ではなく、そのような蓄積はないので絶対に Fortran を使わなければいけない環境ではありません。

Fortran にはポインタなどの概念が存在せず、比較的コーディングしやすいのも利点です。数値計算以外には基本的に向いていないのがデメリットで、Fortran ができるよりも C++ ができる人のほうが、今後数値計算以外のことを仕事にする場合に融通が利くのではないかと考えています。

1.1.3 Python3 について

実のところ筆者が一番長く触れている言語です。数値計算を補助する言語として Python3 も非常に有用です。C++ や Fortran に比べると、Python3 はライブラリが充実していて、可視化やデータ解析を迅速に行える利点もあります。¹ 型指定もないので、簡単な計算を行いたいのなら C++ や Fortran よりも圧倒的に便利です。特に配列 (リスト) の定義が

¹可視化には matplotlib, 数値計算やデータ解析には numpy や scipy といったライブラリが便利でしばしば使われます。

楽で、線形代数関連のライブラリが充実しており簡単に計算できるのは大きな魅力です。コードの読解も比較的やさしく、精神的な負担が少ないです。

Python3 はインタプリタ言語であり、実行速度やメモリ制御の面ではコンパイル型言語に劣るために、大規模数値計算の中核を担うのには不適切である場合もあります²。また型宣言がないのは実装や保守性の観点から苦しみ種となる場合もあります。しかしながら、C++やFortran で実行した本計算の出力を Python で読み込み、プロットや解析を行うといった使い方は広く行われており、その自由度はほかの言語の追随を許しません。

1.1.4 近年注目されている言語 (Julia)

筆者は常用しておらず、軽く触れる程度しかできませんが、近年数値計算を目的としたプログラミング言語として注目されているものの一つに Julia があります。素早く実装することができ、かつ可読性が高い部分は Python に似ていますが、型安定なコードなら C や Fortran に劣らない速度で計算できるため、両者のいいとこどりをしています。現時点でのデメリットといえば、比較的新しい言語であるためにライブラリやパッケージの進化が早くバージョン管理が大変であろうことだと考えられますが、最近は研究会なども開かれており今後利用者は増えていくと見込まれます。

1.2 C++で使える高速数値計算ライブラリ

行列の積は添え字をループで回して和を取れば計算できますし、フーリエ変換もただの数値積分として実装することは一応できます。しかしそれらを自前で実装するのは時間がかかりますし、何より思わぬバグを招いてなかなか研究が進まないといった問題に直面します²。物性理論で頻出するこうした計算は、標準的なライブラリを呼び出して使うのが一般的です。代表的なものを以下に列挙しますが、実装例はのちの章で軽い文法とともに解説します。

1.2.1 BLAS/LAPACK(線形代数用ライブラリ)

BLAS はベクトルおよび行列に対する基本的な演算を提供する数値線形代数のライブラリで、ベクトルの内積をはじめ行列とベクトルの積、行列と行列の積も計算することが

²厳密には計算効率を上げるための工夫が様々に存在するようですが、その工夫に割くコストを考えると最初から C++や Fortran で実装した方がよいという意見が専門家の間では散見されます。

²勉強のために一から実装するのは効果的ではありますが、修士博士を途中まで経た身からすると、そうしたことにかけけている時間はそれほどなくお勧めできません

出来ます。これらの演算はそれぞれ Level 1, 2, 3 のように分類されています。LAPACK は BLAS を基礎として、行列の対角化や連立一次方程式の求解などを行うことが出来ます。LU 分解や QR 分解などの行列の分解を行うことも出来ます。これらの外部ライブラリは長い間使われてきた実績も相まって業界のデファクトスタンダードとなっています。

1.2.2 FFTW3(フーリエ変換)

高速フーリエ変換 (FFT) は読んで字のごとく、フーリエ変換を高速に行うためのライブラリです。多次元のフーリエ変換も行うことが出来ます¹。関数にはいろいろあり、実数関数と複素関数で使い分ける必要があります。また逆フーリエ変換とフーリエ変換の係数の違いは反映していないので、こちらで指定する必要があります。

1.2.3 OpenMP(並列計算)

ループ計算を行うとき、メモリ並列を行って複数の計算を同時に行うことで時間が短縮できることがあります。その手段として OpenMP が用いられます。並列化には MPI という手段もありますが、MPI はプロセスごとに情報の通信を行う必要があり、やや学習コストが高いものとなっています。スレッド並列なので既存のコードをほとんど変えずに並列化可能なのが利点とも言えます。

1.3 計算環境の準備

数値計算に使うツールの概観を終えたので、次に環境構築に移ります。本稿では Windows Subsystem for Linux(WSL) を導入して、仮想環境を構築します。理由の第一は筆者が使い慣れているために詳細が説明できることですが、他の利点としては、ソースコードの編集やコンパイル、実行、結果の整理といった一連の作業をコマンドラインで容易に行うことができる点があります。このような開発を **CUI** ベースといいます。パラメータを変えながらプログラムを連続して走らせたりする場面では、シェルスクリプトや Makefile を用いた自動化も用いられ、その際は CUI 前提の環境の方が圧倒的に相性が良いです。また物性研の計算サーバー (ohtaka, kugui) や東大情報基盤センターの Wisteria などを利用する際はリモート接続による CUI の操作が基本となります。そのため、CUI に慣れておくことは研究を継続するうえでも大切になるでしょう。

¹昔はメッシュサイズが 2 の冪でないと計算できなかったようですが、アルゴリズムの進歩により任意のメッシュサイズで計算できるようになっています。

PC が支給されるとき、最初から OS が Linux になっていることは少なく、Windows か Mac であることが多いと思います。WSL を用いると、Windows 上に仮想的な Linux 環境を導入することで、Linux と同様の感覚でプログラムを開発・実行することが出来ます。これにより Windows を日常的に使いながら、研究で必要な Linux 環境を整えることが出来ます。また Visual Studio Code の Remote と組み合わせることで、エディタを Windows 側で動かしつつ、実際のコンパイルや実行を Linux 側で行うというある種分業的な使い方が出来ます。

1.3.1 WSL の有効化

WSL の導入は Powershell から行います。まずスタートメニューから Powershell と検索し、管理者として実行を選択して起動します³。起動後、以下のコマンドを入力します。

```
wsl --install
```

このコマンドを実行すると、WSL 本体が有効化および Linux ディストリビューションのインストールが自動的に開始されます。⁴

ここで一度 `wsl -l -v` を実行してみます。デフォルトでは Ubuntu の最新バージョンが入っていると思われますが、もしそのほかのバージョンやディストリビューションを指定してインストールしたければ、

```
wsl --list --online
```

でディストリビューションの一覧を確認し、

```
wsl --install -d <Distro>
```

で好みのものをインストールしてください。以下では Ubuntu を導入した場合を仮定します。

WSL の導入が完了すると、スタートメニューから Ubuntu と検索することで、Linux 環境を起動することが出来ます⁵。Ubuntu の初回起動画面ではユーザー名およびパスワードの設定を求められます。ここで設定したユーザー名とパスワードは、管理者権限を使用するときに必要なため、忘れないようにしてください。設定が一通り終わると黒い画面のターミナルが表示されるので、コマンド入力可能な状態になります。以降は Powershell ではなく、bash と呼ばれる Linux 環境のターミナルを用います。

まずパッケージ管理システムを最新の状態に更新する必要があるので、ターミナル上で

³何もせずクリックするとユーザー権限で開かれ、wsl が有効にならないことに注意。

⁴インストール途中で再起動を求められる場合があります。

⁵あるいは、Powershell から `wsl` と打ち込むことでも起動できます。

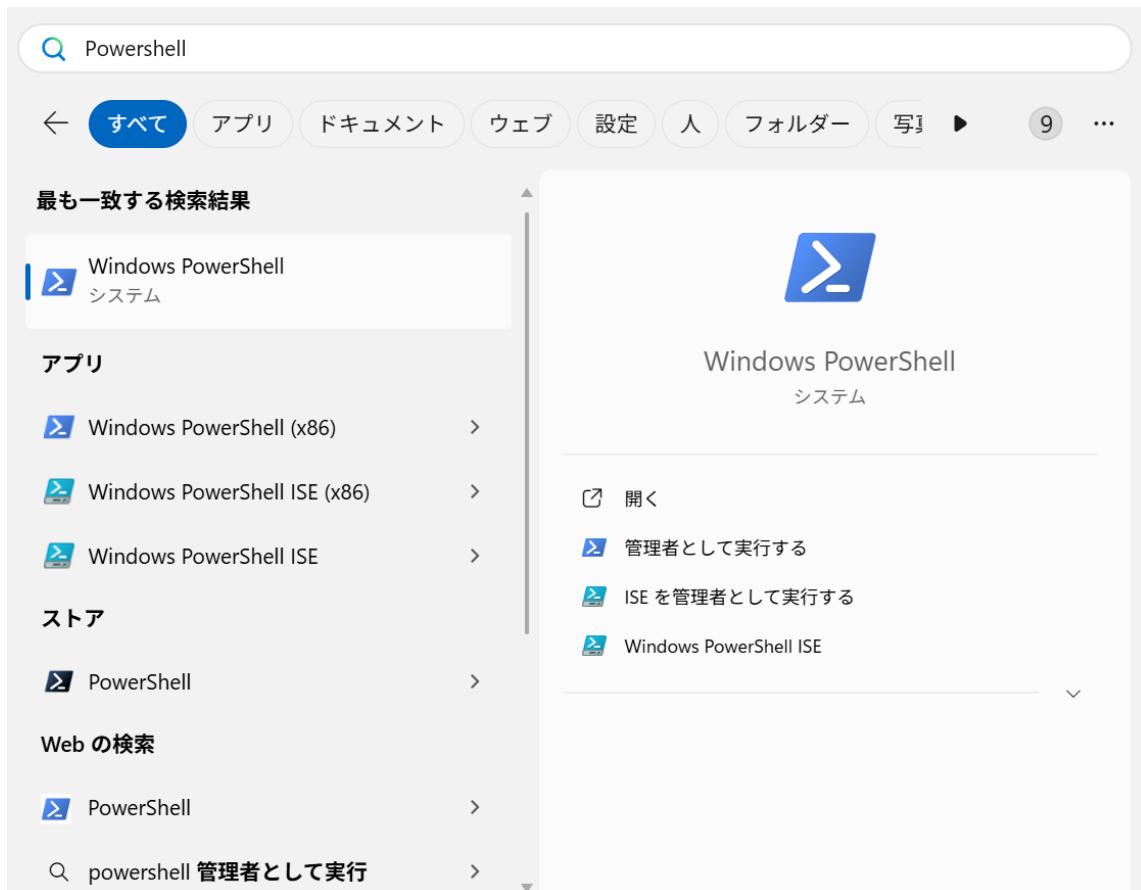


図 1.1: スタートメニューから Powershell と検索するとこのような画面が出てきます。最も一致する検索結果に表示されている Windows PowerShell をクリックして起動してください。

```
sudo apt update
sudo apt upgrade
```

を実行してください。sudo とは、管理者権限でコマンドを実行するための命令で、実行時には設定したパスワードの入力のみが求められます。

1.3.2 Visual Studio Code の導入

次にプログラムの編集および開発に用いるエディタとして、Visual Studio Code(VS Code)を導入します。VS Code は軽量で拡張性が高いこと、また Remote により WSL 環境との連携も容易であるために、本稿では標準的な開発環境として採用します。⁶ここで重要なのが、VS Code は Windows 側にインストールするということです。WSL から sudo

⁶ssh など大学のスパコンを用いる場合、標準装備されているエディタが emacs である場合もあります。

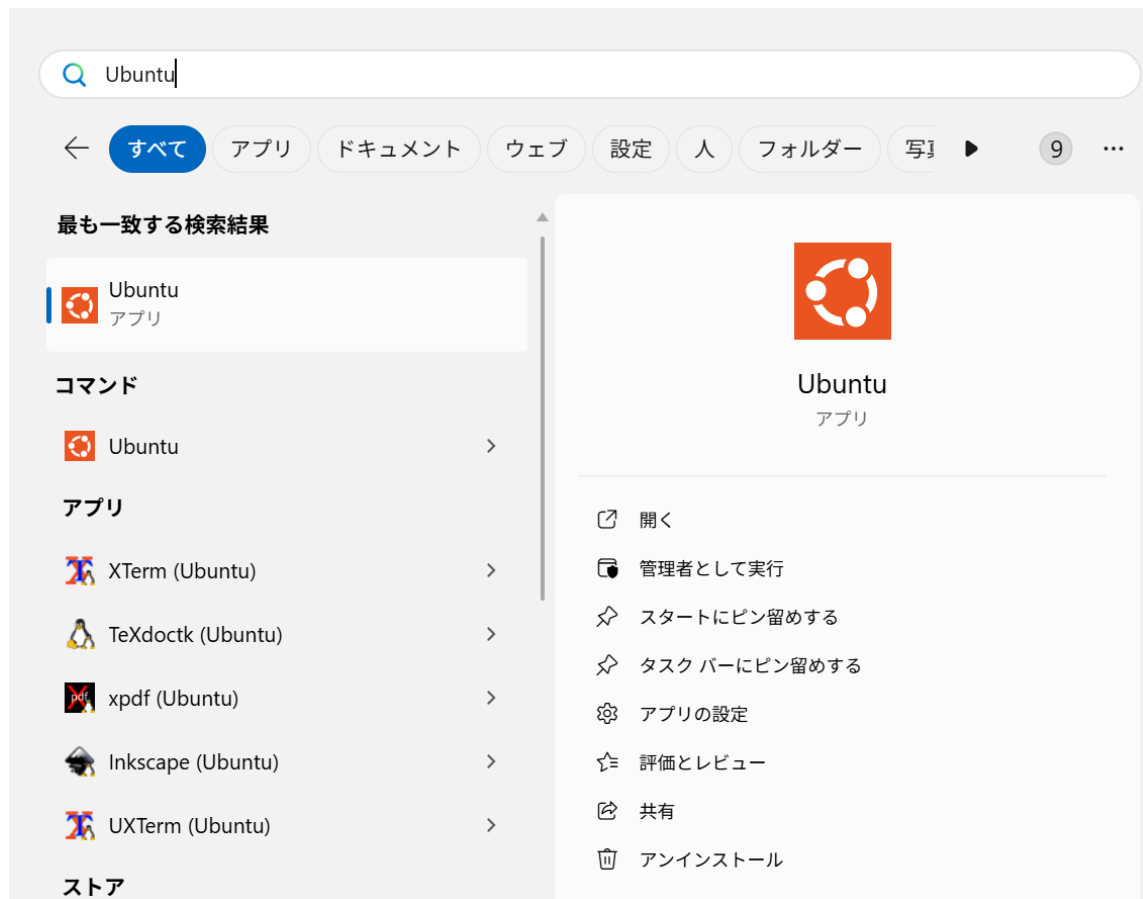


図 1.2: WSL の導入に成功した場合、スタートメニューから Ubuntu と検索するとこのような画面が出てきます。

apt install code してエディタを導入することもできますが、Windows 上で動作する VS Code から Linux 環境に接続することで安定した開発環境を構築できます。

まず、公式サイトから VS Code をダウンロードします。

<https://code.visualstudio.com/>

ブラウザで上記ページを開き、Download for Windows を選択してインストーラを取得します。ダウンロードしたインストーラを実行し、画面の指示に従ってインストールを進めてください。インストールの途中で

- PATH に環境変数を追加する

にチェックをしてください。コマンドラインから VS Code を起動できるようにするために必要です。

インストールが完了したら、スタートメニューから VS Code を起動します。正常に起動するとエディタ画面が表示されると思います。VS Code が正しくインストールされて

いることを確認するためには、Windows 側の Powershell またはコマンドプロンプトを開いて、

```
code --version
```

を入力します。バージョン番号が表示されれば VS Code の導入は正常に完了しています。

次に VS Code の Remote - WSL を用いて、Windows 上のエディタから Linux 環境へ接続する方法について説明します。Remote - WSL は VS Code から WSL 上の Linux 環境に直接アクセスし、ソースコードの編集・実行・デバッグを行うための拡張機能です。これにより、エディタは Windows で動作しつつ、実際の開発環境を Linux で行うという環境が実現します。

まず VS Code を起動し、左側のメニューから「拡張機能 (Extensions)」アイコンを選択します。検索欄に Remote WSL と入力すると、拡張機能が表示されるのでこれをインストールします。⁷

次に Ubuntu のターミナルから VS Code を起動します。Ubuntu を起動して任意の作業ディレクトリに移動した後、以下を実行します。

```
code .
```

. は現在いるディレクトリを指し、ここで code を実行するという意味です。このコマンドを実行することで VS Code が起動し、現在のディレクトリがワークスペースとして開かれます。同時に画面左下に「WSL : Ubuntu」と表示され、Linux 環境に接続されていることが確認できます。この状態では VS Code で編集しているファイルはすべて WSL 側の Linux ファイルシステム上に存在していて、コンパイルや実行も Linux の環境で行われます。⁸

VS Code にはターミナルが内蔵されていますが、WSL に接続された状態で VS Code を開くと、内蔵ターミナルも Linux 環境として動作します。VS Code 上で **Ctrl + `** を押すとターミナルが開きます。ここで表示されるシェルが Ubuntu の bash になっていれば正しく接続されています。⁹

⁷ コマンドパレットを用いてインストールすることもできます。VS Code 上で **ctrl+Shift+P** を押し、**Extensions:install Extensions** と入力して検索を書けることもできます。

⁸ 画面左下に WSL : Ubuntu と表示されていない場合は、左下の緑色のアイコンをクリックして「Connect to WSL」を選択してください。

⁹ 内蔵ターミナルはエディタ内で開発を完結させることが出来て便利ですが、ログが見にくくなったりする欠点もあります。必要に応じて使い分けてください。

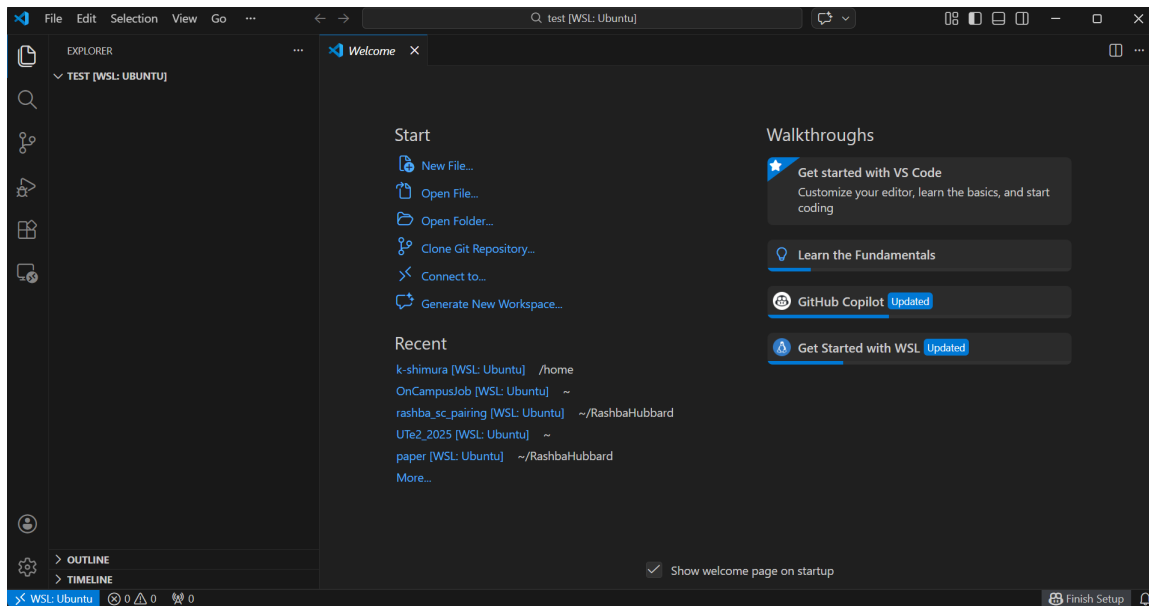


図 1.3: ここでは test というディレクトリを用意し、そこで `code .` を実行することでエディタを開いています。まだファイルはありませんが、ファイルおよびディレクトリを作成すると左側の画面に一覧として表示されます。なお左下が WSL: Ubuntu となっていれば WSL と VS Code を紐づけることが出来ています。

1.3.3 ファイル操作のコマンド

ファイル操作はターミナルを用いた作業においてごく基礎的であるので、一度ここで整理しておきます。基本的にカーソルを用いることはなく、全ての動作がキーボード入力で完結します。最初は少し混乱することもあります。慣れれば GUI の操作よりずっと効率的になります。

まず、今自分がどこにいるのか (`pwd`)、その場所に何があるか (`ls`) をターミナルで打ち込んでチェックすることが出来ます。 `pwd` を入力すると現在いる場所のパスが表示されます。また `ls` コマンドを打ち込むと、現在いるディレクトリに含まれるファイルおよびディレクトリが表示されます。隠しファイル込みでチェックしたい場合は (`ls -la`) を行います。

次に作業ディレクトリを移動するための `cd` コマンドについて説明します。 `cd` はパスを指定して移動するのが基本ですが、ショートカットも便利です。例えば `cd ..` は一つ上の階層への移動、 `cd ~` はホームディレクトリへの移動を表します。また `cd -` は直前にいたディレクトリに戻る操作で、2つの場所を行き来しながら作業するときに便利です。¹⁰

その他、ディレクトリの作成 (`mkdir`)、ファイルの作成 (`touch`)、コピー (`cp`)、削除

¹⁰めったにありませんが、ディレクトリ名にスペースが含まれる場合は `cd "hoge hoge"` のようにクォーテーションで囲む必要があります。

(rm) などがあります。表 1.3.3 にファイル操作でよく使うコマンドを一覧としてまとめています。

表 1.1: ファイル操作でよく使うコマンド

コマンド	概要
pwd	作業中のディレクトリ（カレントディレクトリ）を表示する。
ls	ディレクトリ内のファイル一覧を表示する。ls -l（詳細表示）、ls -a（隠しファイル含む）、ls -lh（サイズの表示）がよく使われる。
cd	ディレクトリを移動する。cd ..（1つ上の階層へ移動）、cd ~（ホームへ移動）、cd -（直前作業していたディレクトリへ移動）等をよく使う。
mkdir	新しいディレクトリを作成する。mkdir -p a/b/c で途中の階層もまとめて作れる。
touch	新しい空ファイルを作る。
cp	ファイル/ディレクトリをコピーする。ディレクトリは cp -r dir1 dir2 とすれば、「dir1 を dir2 以下にコピーする」ことになる。 ¹¹ 。
mv	ファイルの移動または名前変更を行う。mv old new で名前変更。
rm	ファイルを削除する。rm -r でディレクトリ削除復元が基本できないので慎重に使う。 ¹²
cat	ファイルをそのまま表示する。短い設定ファイル確認に便利だが長文には不向き。
less	長いファイルをページング表示する。/ で検索、q で終了。
find	名前や更新日時などの条件でファイルを探す。例: find . -name "*.cpp"。

1.3.4 シェルスクリプト

ターミナル操作に慣れてくると同じ処理を何度か繰り返す場面が増えます。例えばパラメータを変えて同じ計算を繰り返す、複数のファイルを所定の場所にコピーすることがしばしばあります。こうした繰り返し処理はシェルスクリプトや Makefile を使って自動化すると、作業の効率が大きく向上します。

シェルスクリプトはターミナルで打つコマンドを1つのファイルにまとめ、いつでも同じ手順を再実行できるようにするための仕組みです。拡張子は sh です。先頭に shebang と呼ばれるコマンド（#!/usr/bin/env bash）を書いてから、実行したいコマンドを上から書いていきます。詳細は省略しますが、変数の定義のみならずループ、条件分岐なども行うことが出来るので、実験や計算の一括実行が簡単になります。

以下にシェルスクリプトの一例 (sample.sh) を示します。変数 OUTDIR には指定のディレクトリを、変数 BINARY には実行ファイルの名前が入ります。PARAM_LIST にはパラメータをリストとして格納し、後ほどの for 文で繰り返し処理します。変数 log は、変数 tag を受け取ってログファイル名を受け取り、シェルスクリプト実行下で出力します。その他細かい文法については適切な参考書等を参照してください。

sample.sh

```
1  #!/usr/bin/env bash
2
3  OUTDIR="results"
4  BINARY="./main"
5
6  PARAM_LIST=(0.1 0.2 0.5 1.0)
7
8  mkdir -p "$OUTDIR"
9
10 echo "[INFO] build start"
11 make -j
12 echo "[INFO] build done"
13
14 timestamp="$(date +%Y%m%d_%H%M%S)"
15
16 for a in "${PARAM_LIST[@]"; do
17     tag="a_${a}"
18     log="$OUTDIR/${timestamp}_${tag}.log"
19
20     echo "[INFO] run: a=${a} -> ${log}"
21     "${BINARY}" --a "${a}" > "${log}" 2>&1
22 done
23
24 echo "[INFO] all runs completed."
```

1.3.5 Makefile

実行したいプログラムが1ファイルであれば、毎回ターミナルから手でコンパイルコマンドを打っても大した面倒ではないのですが、ソースコードが複数ファイルに分かれてきたり、複数のライブラリを併用するようになると、毎回長いコマンドを入力するの

は手間がかかるうえ、オプションの付け忘れも起こりやすくなります。そこでよく用いられるのが Makefile です。

Makefile とは、どのファイルからどのように実行ファイルを作るかを記述した設定ファイルです。Makefile を作成しておけば、以後は

```
make
```

と入力するだけでコンパイルを実行できます。また、変更のあったファイルだけを再コンパイルしてくれるため、ソースコードが大きくなった場合にも便利です。

最も簡単な Makefile の例を以下に示します。

Makefile

```
1 CXX = g++
2 CXXFLAGS = -O3
3 TARGET = main
4 SRC = main.cpp
5
6 $(TARGET): $(SRC)
7 ^^I$(CXX) $(SRC) $(CXXFLAGS) -o $(TARGET)
8
9 clean:
10 ^^Irm -f $(TARGET)
```

ここで、CXX はコンパイラ、CXXFLAGS はコンパイル時のオプション、TARGET は出力される実行ファイル名、SRC はソースファイル名を表しています。

この Makefile を Makefile という名前で保存しておけば、

```
make
./main
```

とすることでプログラムを実行できます。また、

```
make clean
```

とすると、生成された実行ファイルを削除できます。

OpenMP や FFTW、OpenBLAS などを使う場合は、ライブラリやオプションを CXXFLAGS や別の変数にまとめて書くと見通しがよくなります。例えば、OpenMP、FFTW、OpenBLAS を同時に用いる場合は次のように書けます。

Makefile

```
1 CXX = g++
2 CXXFLAGS = -O3 -fopenmp
```

```

3 LIBS = -lfftw3_omp -lfftw3 -lopenblas -lm
4 TARGET = main
5 SRC = main.cpp
6
7 $(TARGET): $(SRC)
8 ^^I$(CXX) $(SRC) $(CXXFLAGS) $(LIBS) -o $(TARGET)
9
10 clean:
11 ^^Irm -f $(TARGET)

```

このようにしておく、コンパイル条件を変更したいときも CXXFLAGS や LIBS を書き換えるだけで済みます。

ソースコードが複数ファイルに分かれている場合には、Makefile の利点がさらに大きくなります。たとえば main.cpp, solver.cpp, util.cpp の3つのファイルから実行ファイルを作る場合は、以下のように書けます。

Makefile

```

1 CXX = g++
2 CXXFLAGS = -O3 -fopenmp
3 LIBS = -lfftw3_omp -lfftw3 -lopenblas -lm
4 TARGET = main
5 SRCS = main.cpp solver.cpp util.cpp
6 OBJS = $(SRCS:.cpp=.o)
7
8 $(TARGET): $(OBJS)
9 ^^I$(CXX) $(OBJS) $(CXXFLAGS) $(LIBS) -o $(TARGET)
10
11 %.o: %.cpp
12 ^^I$(CXX) -c $< $(CXXFLAGS) -o $@
13
14 clean:
15 ^^Irm -f $(TARGET) $(OBJS)

```

この例では、各 .cpp ファイルから対応する .o ファイルを作り、それらを最後にまとめてリンクしています。ファイル数が増えても管理しやすく、変更のあったファイルだけが再コンパイルされるため効率的です。

Intel コンパイラや MKL を用いる場合も、Makefile 中の変数を書き換えるだけで対応できます。例えば、

Makefile

```
1 CXX = icpc
2 CXXFLAGS = -O3 -qopenmp -qmk1
3 TARGET = main
4 SRC = main.cpp
5
6 $(TARGET): $(SRC)
7 ^^I$(CXX) $(SRC) $(CXXFLAGS) -o $(TARGET)
8
9 clean:
10 ^^Irm -f $(TARGET)
```

のように書けば、MKL を利用したコンパイルを `make` で実行できます。

1.4 GitHub の導入 (やや発展)

研究で書いたコードは、実験ノートのように変更履歴を後から追えると作業の面でも研究倫理の面でも望ましいのですが、実際に研究を行ってみると、ファイル名が増殖したり、最新版が分からなくなったり、どの変更でどう結果が変わったのかわからなくなったりします。また最悪のケースだと誤って上書きしてしまったり削除してしまったりする事故も起こり得ます。ここで GitHub が便利です。GitHub とは、Git¹³ を用いて管理しているリポジトリをインターネット上で保管あるいは共有するためのサービスで、共同開発の現場で広く使われているものですが、個人のプロジェクトの管理においても便利です。ここでは必要最小限の機能を紹介し、リポジトリの作成方法や基本的な操作などを紹介します。

とはいえ紙面と時間の都合により詳しく述べられない部分もあるので、必要な情報は公式ドキュメントや適当な書籍などで補ってください。

1.4.1 インストール

まず `git` が有効か確認します。WSL のターミナルで

```
git --version
```

と打って `git` のバージョンが表示されれば導入済みです。未インストールの場合は

¹³履歴を記録しながらファイルを管理する仕組みのこととってください。

```
sudo apt update
sudo apt install -y git
```

でインストールしましょう。

1.4.2 履歴の残し方

まずリポジトリを作ります。作業したいプロジェクト用のディレクトリに移動して

```
git init
```

とコマンドを打ちます。これによってディレクトリ内部に.git/が作成され、以後そのフォルダはリポジトリとして履歴を持つようになります。

コードを書いたりしてファイル内容に変更が生じた場合は、その変更内容をステージに載せます。ステージは更新内容の候補のようなものだと思います。

```
git add .
```

この変更をメッセージ付きで確定するのがgit commitです。

```
git commit -m "add files for RPA calculation"
```

コミットメッセージは後で見返したときにどんな変更をしたのかわかるように書くといいです。とはいえあまり長く書いてもまどろっこしいので簡潔にまとめます。

1.4.3 SSH 接続

リポジトリに変更を反映すること、リポジトリの変更をローカルにも反映することはそれぞれgit push, git pullで行うことができますが、この作業には事前にGitHubとSSHを用いて通信できるようにしておく必要があります。

SSHは暗号化によって安全な通信を実現する仕組みのことです。秘密鍵と公開鍵の2つを用意し、秘密鍵を自分のPCに保持して公開鍵の方をGitHubに登録しておきます。

事前確認として、まず

```
ls -la ~/.ssh
```

を試してみてください。最初に何もしていなければ空っぽだと思うので、次の手順で作成します。

SSH 鍵の作成

GitHub に SSH 接続するための鍵を作成します。

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

このコマンドを実行すると保存先及びパスフレーズ¹⁴を聞かれるので適宜入力してください。保存先は通常そのまま Enter キーを押すとデフォルトで `/.ssh/id_ed25519` に格納されます¹⁵。基本的にはデフォルトの場所に格納するのをお勧めします。

公開鍵の登録

次に GitHub の Web サイトに移動し、以下のように設定してください。GitHub のマイページでアイコンをクリックすると、Setting というタブがあるので、それをクリックしてください。SSH and GPG keys から New SSH key をクリックします。Title と key を求められますが、Title は自由に設定してください。ssh key の入っている端末がわかるような名前がよいでしょう。また key には先ほど作った公開鍵 `id_ed25519.pub` の中身を貼ってください。

登録が出来たら SSH 接続ができるか試してみましょう。

```
ssh -T git@github.com
```

初回は接続先を信頼するかのメッセージが出ると思うので、内容を確認したうえで `yes` と入力してください。以下のようなメッセージが表示されれば成功です。

```
Hi <username>!  
You've successfully authenticated, but GitHub does not  
provide shell access.
```

1.4.4 リポジトリへの変更の反映

`git push` は、ローカルで作成した変更履歴を GitHub へ送信して反映する操作です。push で送られるのは原則コミット済の変更なので、ファイルを編集しただけでは GitHub には反映されません。

個人で作業を進める場合は、きちんと `add`, `commit`, `push` していれば手元のファイルの更新と GitHub 上の履歴が一致しているはずですが、会社などで集団で作業する場合は他

¹⁴あまり大きい声では言えないが、ここでのパスフレーズはきちんと設定しなくても大丈夫で、空のほうが作業効率は向上します。

¹⁵`ed25519` は鍵を作るときのアルゴリズムに関連した名前らしいですが、筆者はあまり詳しくありません

者の変更を共有して自身の環境にも反映させる必要があるので `git pull` してリポジトリの情報をローカルに反映してください。

1.4.5 その他

ここまで紹介してきたコマンドは Git に標準搭載されているもののほんの一部に過ぎず、実際の作業現場ではブランチという作業場所の切り替え、`config` の設定など大量の内容があります。ここで全てを解説することは筆者の能力と時間の観点から難しいので、少なくとも `git add -A`, `git commit -m "hogehoge"`, `git push` を覚えておいてください。

| 1.5 C++ / Python 開発環境の構築

ここまでで WSL 上の Ubuntu と VS Code を連携させ、Linux 環境上でプログラムを編集及び実行できる状態が整っています。以降は、本稿で扱う数値計算を行うために必要となる C++ および Python の開発環境を構築します。

1.5.1 インストール

まずは C++ 及び Fortran のコンパイルに必要な基本的なツール群を導入します。ターミナル上で以下のコマンドを実行してください。

```
sudo apt install -y build-essential gfortran make cmake
```

すると `g++`, `gcc`, `gfortran` といったコンパイラがインストールされます。これらはそれぞれ C++, C, Fortran のコンパイラに対応し、作ったプログラムをコンパイル (機械語への翻訳) して実行ファイルを作成するために必要です。また `make` や `cmake` はビルドの管理・設定に必要なツールとなっています。

次に主要な数値計算のライブラリをインストールします。

```
sudo apt install -y libblas-dev liblapack-dev libfftw3-dev
```

これにより、BLAS/LAPACK (線形代数の演算に必要なライブラリ)、FFTW3 (高速フーリエ変換に必要なライブラリ) がインストールされます。なお並列化に必要な OpenMP は `g++` に標準で組み込まれており、コンパイル時に `-fopenMP` とオプションを追加することで使用できます。

引き続き Python 環境の構築も行います。ターミナルで以下を実行してください。

```
sudo apt install -y python3 python3-pip python3-venv
```

Python 本体とパッケージ管理ツールが導入されます。次に数値計算・可視化で頻繁に用いられる `numpy` や `scipy`, `matplotlib` といった主要なライブラリを導入します。

```
pip3 install --user numpy scipy matplotlib
```

数値計算に使うライブラリは特に NumPy と SciPy です。NumPy は行列演算やフーリエ変換のみならず、かなりオールマイティな数値計算ライブラリです。SciPy は微分方程式を解く際などにしばしば用いられます¹⁶。Python の良いところはデータを可視化するためのライブラリも充実していることです。特にグラフ描画などに Matplotlib が使われます。

1.5.2 動作確認

環境構築が成功しているかを確かめるために、Hello World を行います¹⁷。まず C++ が動作するかをたしかめます。chapter1-1.cpp はターミナルに”Hello World!”を標準出力させるプログラムです。

chapter1-1.cpp

```
1 #include <iostream>
2
3 int main(){
4     std::cout << "Hello World\n";
5     return 0;
6 }
```

以下のコマンドで実行することができます。

```
g++ chapter1-1.cpp -o main
./main
```

ターミナルに”Hello World!”が出力されれば成功です。

次に Python3 の動作確認をします。Python3 はファイルを作っても良いのですが、ターミナルから以下を直接うちこむことでも”Hello World”の出力を行うことができます。

```
python3 -c 'import numpy, scipy, matplotlib;\nprint("Hello World!")'
```

¹⁶筆者はあまり使ったことがありませんが……。

¹⁷説明するのは野暮ですが、プログラムが正しく動作するかを確かめるために、”Hello World”を標準出力させる文化があります。

Chapter 2

C++による数値計算の基礎

「基礎」と銘打っていますが本章ではC++の基礎的な文法について既知とし、研究のきっかけに最低限必要な機能とその使い方について解説します。C++の文法についてほとんど知らないという方は適宜勉強するのも良いと思いますが、ここに書いてあることをとにかく実行して手を動かしてみるのも一つの手です。なお Fortran や Python、Julia による実行は今回は割愛いたします。

2.1 コンパイルおよび Makefile の作成

1 章にもあったように、最もシンプルなコンパイル方法は

```
g++ chapter1-1.cpp -o main
./main
```

です。ここでは chapter1-1.cpp が入力ファイル、-o の後に指定しているものが出力ファイルの名前です。このように C や C++ のプログラムのコンパイルは通常ソースファイルと出力ファイルの名前を指定するだけでよいのですが、FFTW3 や LAPACK/BLAS といった外部のライブラリを利用する際には、コンパイル時にオプションを指定する必要があります。

blas を利用したプログラムをコンパイルしたい場合は

```
g++ main.cpp -O3 -lopenblas -o main
```

のように扱います。ただし物性研のスパコンなどを使う場合は Intel MKL 環境が blas をサポートしているので

```
icpc main.cpp -O3 -qmk1 -o main
```


のように入力します。

ここで-O オプションは最適化のレベルを指定するオプションで、-O0, -O1, -O2, -O3 といった種類があります。末尾の数字が大きいほど最適化のレベルが高くなる一方で、コンパイルするための RAM が増大する側面もあります。¹

FFTW を利用したプログラムをコンパイルしたい場合は

```
g++ main.cpp -O3 -lfftw3 -lm -o main
```

とします。ただし FFT を並列計算で扱う場合は新たに追加で-lfftw3_omp も必要です。

OpenMP を用いる場合は-fopenMP をオプションに付加すればよいです。これらを複合的に用いる場合は、コンパイル時にまとめてオプションを付けます。例えば

```
g++ main.cpp \  
-O3 \  
-fopenmp \  
-lfftw3_omp \  
-lfftw3 \  
-lopenblas \  
-lm \  
-o main
```

のようにします。

2.2 C++による対角化

本節では、C++から LAPACK を呼び出してエルミート行列を対角化するための手順をまとめます。複素行列を扱うか実対称行列を扱うかで用いる LAPACK のルーチンが異なる点にも注意すべきですが、呼び出し自体は数行で済みます。対角化には、エルミート行列を対角化する zheev か、dsyev を用います。LAPACK はもともと Fortran で用いられていたライブラリであるため、column-major でメモリの連続領域に格納されています。そこで C++では `std::vector<std::complex<double>>` を用いることで、要素 H_{ij} に $H[i + N * j]$ としてアクセスすることにします。

サンプルプログラムは `chapter2-1.cpp` です。

¹基本的に-O3 を使えばよいですが、コンパイルが通らなくなる場合は-O0 を使います。それでも通らない場合はファイルを分割するなどの工夫が必要になります。

2.2.1 zheev の引数

zheev の引数はこちらが考えて指定するものはそれほど多くなく、慣れれば簡単に実装できるようになりますが、一応各引数の説明をします。

表 2.1: LAPACK 固有値計算ルーチンの主な引数

引数名	型	概要
jobz	char	'V' または 'N' を指定する。'V' では固有値と固有ベクトルを計算し、'N' では固有値のみを計算する。
uplo	char	'U' または 'L' を指定する。エルミート行列の上三角部分または下三角部分のどちらを参照するかを指定する。
n	int	対角化する行列のサイズ。 $n \times n$ 行列の場合は n を指定する。
a	std::vector<complex<double>>	対角化したい行列の成分を格納する配列。
lda	int	先頭次元。通常は n でよい。 n より小さい値を与えると誤動作の原因となる。
w	std::vector<double>	固有値を昇順で格納する配列。エルミート行列の固有値は実数である。
lwork	int	作業配列サイズ取得用。最初に -1 を指定して必要サイズを問い合わせる。
work	complex<double>*	lwork で取得したサイズに基づいて確保する作業配列。
rwork	std::vector<double>	実数作業配列。通常は $3n - 2$ を指定する。
info	int	計算の終了状態を示す。 0 なら正常終了、 0 以外はエラーを示す。

zheev は入力で与えた行列を上書きするので、元のハミルトニアンを別に使いたい場合は、対角化前に行列をコピーして保持しておくべきです。

2.2.2 コンパイルおよび実行

LAPACK は単体では動かず、内部で BLAS も呼び出しています。したがって、C++ で LAPACK を使う場合、リンク時に LAPACK と BLAS を同時にリンクする必要があります。

す。コンパイルは

```
g++ chapter2-1.cpp -O2 -llapack -lblas -o main
```

のように行います。無事コンパイルが終わると実行ファイル `main` が生成されるので、

```
./main
```

で実行できます。

2.2.3 対角化の手順

`chapter2-1.cpp` は、 3×3 のエルミート行列

$$H = \begin{pmatrix} 1.0 & 0.2 + 0.1i & 0 \\ 0.2 - 0.1i & 2.0 & 0.3 \\ 0 & 0.3 & 3.0 \end{pmatrix} \quad (2.1)$$

を対角化し、行列の固有値と固有ベクトルを出力するコードです。また対角化が出来ているかどうかの検証も行っています。

LAPACK の関数を C++ から呼ぶために、`extern "C"` という宣言を行っています。もともと `dsyev` と `zheev` は Fortran で実装されている関数で、C++ から呼び出すのために必要な宣言となっています。¹

またここでは行列を表現する配列を実装する際に `template <class T>` を使うことで、`T` を型パラメータとして、`T` が `double` 型であっても、複素型であっても同じ行列として認識できるようにしています。

では実際にプログラムを実行してみましょう。この行列の固有値は

$$\lambda_1 \approx 0.95028847, \quad \lambda_2 \approx 1.96487426, \quad \lambda_3 \approx 3.08483726.$$

となっていて、解析的にはきれいに求まりません。きちんと対角化できているかをチェックするには、エルミート行列の固有値及び固有ベクトルの性質に問題がないかを確認する必要がありますが、ここではエルミート行列の以下のような性質を用いて検証します。

- 固有値は実数である
- 固有ベクトルは正規直交系をなす

¹C++では name mangling といい、関数の名前を定めるときに、引数の型や返却値の型など複数の意味を含めて修飾する手法があります。C++で参照する名前と Fortran で参照する名前が違う場合に、`extern "C"` とすることで C 言語と互換な名前解決を試みる事が出来ます。

上の2つの性質から、固有ベクトルを並べた行列 V が $V^\dagger V = I$ を満たすことがわかります。

chapter2-1.cpp を実行すると、出力結果は以下のようになります。

```
Eigenvalues (ascending):
  w[0] = 0.950288
  w[1] = 1.96487
  w[2] = 3.08484

Eigenvectors (columns):
v[0] = ( (-0.872671, -0.436335) (0.216909, 0) (-0.0317472, 0) )
v[1] = ( (-0.194332, -0.0971661) (-0.937531, 0) (0.271715, 0) )
v[2] = ( (0.0260935, 0.0130468) (0.272004, 0) (0.961854, 0) )

Max residual norm2: 4.44957e-16
```

要素数3の1次元配列 w に固有値が昇順に格納されています。また規格化された固有ベクトルが1次元配列 v に格納されています。Max residual norm2 は、各固有値 λ_i と対応する固有ベクトル $\mathbf{v}_i$ について $r_i = |\hat{H}\mathbf{v} - \lambda_i\mathbf{v}_i|^2$ を計算し、その最大値として与えています。最大値が 10^{-16} 程度なので、数値誤差の程度で一致しており、対角化の結果が正しいことが保証されています。

2.3 C++によるFFTW3を用いたフーリエ変換

次にFFTWを用いたフーリエ変換の解説に移ります。FFTWは高速フーリエ変換を行うためのライブラリです。大まか流れとしては、まずフーリエ変換したい配列を用意した後、`fftw_plan_*`のように用意されている関数でplanを作成し、`fftw_execute(plan)`でplan(フーリエ変換)を実行、その後planを破壊してメモリを開放する、という流れになります。フーリエ変換後にplanとメモリを開放することを忘れないようにしてください。

2.3.1 FFTWにおける型

FFTWを用いる際は、通常では見慣れない型がいくつか登場します。まずFFTWにおける複素数型は`typedef double fftw_complex[2]`のように定められています。[0]には実部が、[1]には虚部が対応しています。FFTWで扱う引数は、`std::complex<double>`を受け付けないので、FFTの際は複素数型に必ず`fftw_complex`を指定しましょう。

次に、FFTW で要となる `fftw_plan` 型についてです。この型の中身はユーザーからは見えないのですが、入力値や次元数、最適化の仕方やアルゴリズムなどが内部に保持されています。我々の行うことは、フーリエ変換のための `plan` を作成し、それを `fftw_plan` で実行することです。

2.3.2 plan とは何か

FFTW で FFT を実行する場合は、`fftw_plan` 型を持つ変数 `plan` を用意します。例えば 2 次元 DFT を複素関数から複素関数に対して与える場合は

```
1 fftw_plan plan = fftw_plan_dft_2d(...);  
2 fftw_execute(plan);
```

のように実行します。ここで `plan` は変換の次元、各次元のメッシュサイズ、実数か複素数かの型指定、入力・出力の配列、使用する FFT アルゴリズム (順変換か逆変換か) を指定します。なお `plan` を作成し終わった後にユーザーが中身を参照することはできません。

FFTW ではデータ型および変換の種類に応じて使用する `plan` 作成関数が異なります。主な関数は以下のように分類されます。

表 2.2: plan 実行のための関数

関数名	変換の種類	概要
<code>fftw_plan_dft_{1d,2d,3d}</code>	複素数から複素数	複素関数に対する離散フーリエ変換（DFT）を行う。最も一般的に用いられる関数であり、順変換・逆変換の両方に対応する。
<code>fftw_plan_dft</code>	複素数から複素数	任意次元のフーリエ変換を行う。次元数および各方向の格子点数を配列で指定する必要がある。
<code>fftw_plan_dft_r2c_{1d,2d,3d}</code>	実数から複素数	実関数から複素関数へのフーリエ変換を行う。実入力を効率的に扱うため、出力は半分の情報のみを保持する形式になる。
<code>fftw_plan_dft_c2r_{1d,2d,3d}</code>	複素数から実数	複素関数から実関数への逆変換を行う。通常は <code>r2c</code> の逆操作として用いられる。
<code>fftw_plan_r2r_{1d,2d,3d}</code>	実数から実数	離散コサイン変換（DCT）や離散サイン変換（DST）など、実数のみを扱う各種変換を行う。変換の種類は別途指定する。

また FFTW ではデータ型に応じて関数の接頭辞が以下のように異なります。

- `fftw_` : double 型
- `fftwf_` : float 型
- `fftwl_` : long double 型

例えば、複素数に対する 2 次元フーリエ変換を行う場合は、`fftw_plan_dft_2d` を使います。試みに 2 次元フーリエ変換を行う関数がどのような構成になっているか見てみましょう。以下は `fftw_plan plan = fftw_plan_dft_2d( Nx, Ny, in, out, FFTW_FORWARD, FFTW_MEASURE );` の引数の解説です。

表 2.3: FFTW 2次元フーリエ変換ルーチンの主な引数

引数名	型	概要
n0	int	第1次元方向の格子点数を指定する。
n1	int	第2次元方向の格子点数を指定する。
in	fftw_complex*	入力用複素数配列。実部・虚部を分けて格納する必要がある、例えば複素数 a に対して <code>in[ix*n1+iy][0]=a.real(); in[ix*n1+iy][1]=a.imag();</code> のように指定する。
out	fftw_complex*	出力用複素数配列。入力配列の破壊を許容する場合は <code>in==out</code> としてもよい。
sign	int	変換方向を指定する。FFTW_FORWARD は順変換，FFTW_BACKWARD は逆変換に対応する。逆変換後には正規化を行う必要がある。
flags	unsigned int	計算方法を指定するフラグ。FFTW_ESTIMATE および FFTW_MEASURE が利用可能である。

2.3.3 フーリエ変換の手順

chapter2-2.cpp に FFTW を用いて実空間表示の関数を波数空間の関数にフーリエ変換するプログラムを示します。実空間表示の関数として、ここでは2次元格子におけるローレンチアン

$$f(x, y) = \frac{\delta^2}{x^2 + y^2 + \delta^2} \quad (2.2)$$

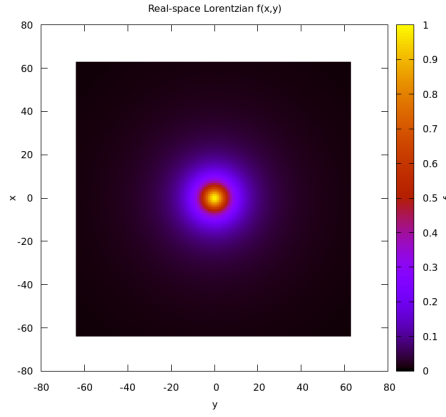
を採用します。ここで $\delta > 0$ はローレンチアンの幅に対応するパラメータで、値を大きくすると実空間で緩やかに減衰します。また、数値計算では、無限平面 $\mathbb{R}^2$ を直接扱うことはできないため、 x, y を有限サイズの正方格子

$$x = -\frac{N_x}{2}, \dots, \frac{N_x}{2} - 1, \quad y = -\frac{N_y}{2}, \dots, \frac{N_y}{2} - 1 \quad (2.3)$$

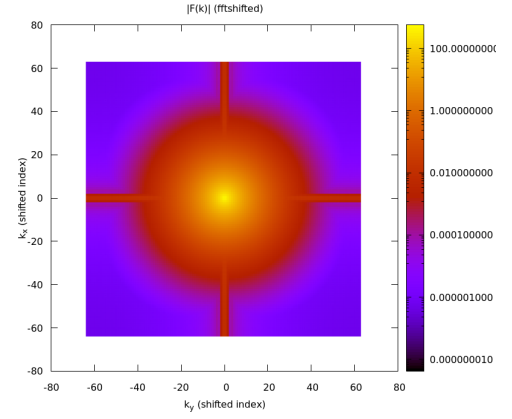
上で定義します。ここでメッシュサイズを $N_x = N_y = 128$ とします。

2次元フーリエ変換

$$F_{m_x, m_y} = \sum_{x=0}^{N_x-1} \sum_{y=0}^{N_y-1} f_{x,y} e^{-2\pi i \left(\frac{m_x x}{N_x} + \frac{m_y y}{N_y} \right)} \quad (2.4)$$

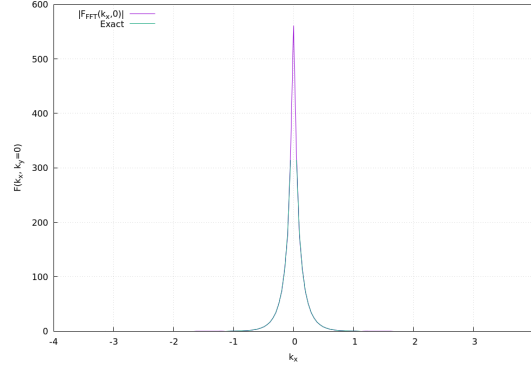


(a) 実空間のローレンチアン $f(x, y)$ のプロット



(b) フーリエ変換後波数空間上にプロットされたローレンチアン

図 2.1: 実空間のローレンチアン $f(x, y)$ と波数空間上の $F(k_x, k_y)$ の比較



(a) (c) compare exact

図 2.2: 厳密解 $F(k_x, k_y)$ と数値フーリエ変換の結果の比較。

を数値計算して波数空間上にプロットします。波数との対応は

$$k_x = \frac{2\pi}{N_x} m_x, \quad k_y = \frac{2\pi}{N_y} m_y \quad (2.5)$$

で与えられます。 m_x および m_y は 0 以上の整数です。

ローレンチアンのフーリエ変換は厳密解が知られていて

$$F(k_x, k_y) = 2\pi\delta^2 K_0(\delta|k|) \quad (2.6)$$

となります。ここで $K_0(x)$ は第 2 種変形ベッセル関数です。ところで $|\mathbf{k}| \rightarrow 0$ の極限では $K_0(z) \sim -\ln z$ のように対数発散するため、 $k = 0$ 成分は数値計算で直接比較できません。そこで本研究では波数空間上の $k = 0$ となる点を除外し、 $|\mathbf{k}| > 0$ の領域において

数値結果との比較を行います。

図 2.2(a) にはフーリエ変換前の実空間のローレンチアンのカラーマップ、(b) にはフーリエ変換後のローレンチアンのカラーマップを示しています。また図 (c) では、 $k_y = 0$ でスライスしたフーリエ変換の実部と厳密解とを比較しています。数値計算結果と厳密解が合致していることが分かります。

2.4 C++による OpenMP を用いた並列計算

OpenMP はメモリ共有によって並列計算を行うための API で、C++ や Fortran のコードに対して並列化を行うことができます。1 つのプロセスの内部で複数のスレッドを生成し、それらが同一のメモリを共有しながら計算を行います。したがって、配列やグローバル変数をそのまま共有して用いることが出来ます。まず parallel region を作り、その中でタスクをスレッドに分配し、各スレッドが同時に計算を実行する、というのが OpenMP による並列化のおおまかな流れです。

2.4.1 いかなるときに並列化が有効か

並列化は、ループの反復が互いに独立であるときに適しています。最も頻繁に使われるのは多重積分でしょう。また、パラメータの掃引においても並列化は威力を発揮します。独立であるかを判断する目安として、並列化させる計算の順序を入れ替えても良いかどうかを検討するとよいです。

もちろんすべてのプログラムが並列化可能なわけではありません。例えば、各 for 文の計算が非常に軽い場合はむしろ並列化のためのオーバーヘッドのほうが負荷になります。また各ループの計算が独立でない場合は意図しない結果をもたらしたり、そもそも並列化不可能であったりします。例えば漸化式のように、 i 番目の計算を出力するのに $i-1$ 番目の結果が必要になる場合は並列化を適用できません。

2.4.2 基本的な使い方

OpenMP はコンパイラが対応している必要がありますが、本稿で主に用いる g++ ではデフォルトで使うことが出来ます。使用時はコンパイルオプションに `-fopenmp` を付ければよく、

```
g++ -O2 -fopenmp main.cpp -o main
```

とします。

対角化やフーリエ変換と比べると、メモリや操作順序を意識しなければならないという点で並列化には別の困難がありますが、実装自体は大変簡単で、以下の3つの指示を覚えれば基本的には事足ります。

- `parallel / parallel for` : `parallel` は並列領域を作成する指示。ほとんど `for` 文と組み合わせて使うので `parallel for` と書くことが多い。
- `shared / private` : 変数を各スレッドごとにコピーするか、全てのスレッドで共有されるものを区別するのに使う指示。
- `reduction` : 複数のスレッドが計算した結果を1つにまとめるために使う。例えば、総和や内積など。

スレッド数は実行時に環境変数を指定すれば変更することが出来ます。例えば実行前に

```
export OMP_NUM_THREADS=4
```

とすれば、OpenMP は4スレッドで実行されます。

2.4.3 `parallel / parallel for`

`chapter2-3-1.cpp` をコンパイルし実行してみましょう。正しい挙動を示せば出力は以下のようなはずです。

```
thread 9 / 16
thread 10 / 16
thread 2 / 16
...
```

まず `#pragma omp parallel` で `parallel region` を作成すると、OpenMP がスレッドを複数作成します。この場合は16本作成されています。`omp_get_thread_num()` がスレッド番号を、`omp_get_num_threads()` が `parallel region` 中のスレッド数を指しています。つまり `thread i / 16` は16本あるスレッドのうち `i` 番目のスレッドが標準出力を実行したことになります。

`chapter2-3-2.cpp` をコンパイルし実行すると、出力は以下のようになります。

```
i = 6, thread = 6
i = 0, thread = 0
i = 7, thread = 7
...
```

このように各スレッドでバラバラに計算が行われます。

2.4.4 OpenMP における変数の扱い (shared/private)

OpenMP では、parallel region に入ったときに各変数が全てのスレッドで共有されているのか、あるいはスレッドごとに独立に定められるのかを意識する必要があります。例えばすべてのスレッドで統一したかった変数が各スレッドで書き換えられてしまい、実行ごとに値が変わってしまうトラブルが起り得ますが、こうした問題を防ぐために変数が shared か private かを定めておく必要があります。shared 変数は全スレッドが同じメモリ領域を参照するので、1つの変数を全員で共有していると考えることが出来ます。一方で private な変数は、各スレッドが専用にコピーを保持し、他のスレッドが参照するのを防ぎます。

簡単な例でその違いを実感してみましょう。chapter2-3-3.cpp は、x に 1 を加算していく操作を並列で行うもので、直観的にはスレッド数が出力されるはずです。しかし結果は実行ごとに異なる値を吐き出します。並列計算では変数をデフォルトで shared としており、計算の競合が発生するためです。¹

chapter2-3-4.cpp では x を private 変数として保持しています。各スレッドで `x += 1` し、全てのスレッドで `x = 1` となっていることがお分かりになると思います。

2.4.5 reduction

OpenMP における shared 変数の扱いを見ると、並列計算には慎重な取り扱いが必要であることが分かります。例えば for 文を使って配列 a と配列 b の内積をとる操作においても、脚注で示したような競合が発生し、正しく総和が取られない可能性があります。そこで導入されるのが reduction です。

reduction は各スレッドに private な変数を用意して、その変数を更新します。並列領域がクローズした後に、各スレッドで更新済みの変数を合計します。これにより競合をさけ、予期しない挙動を防ぎます。

reduction の基本的な構文は `#pragma omp parallel for reduction(operator : variable)` で与えられます。例えば、変数 X に何かの総和を記録するときは、`#pragma omp parallel`

¹実行結果が毎回異なるのは少々発展的な話題であるので脚注にまとめます (しかし大事です!)。まず x に 1 を足すという操作 (`x += 1`) は、CPU の中で複数の手順を通じて行われます。「メモリから x を読み込み、レジスタ上でインクリメントし、結果をメモリに書き戻す」というのが `x += 1` で行われていることです。今、スレッド 1 とスレッド 2 が同時に `x += 1` を実行すると、先にスレッド 1 がメモリから `x = 0` を読み込んだとしても、インクリメントする前にスレッド 2 がメモリから `x = 0` を読み込んでしまいます (本文で「競合」と呼んだ内容)。すると、スレッド 1 が `x = 1` をメモリに書き戻した後に再びスレッド 2 が `x = 1` をメモリに書き込んでしまいます。これにより、スレッド 1 つ分の操作が消えてしまうことになります。つまり運よく x がスレッド数と等しくなることはあるでしょうが、実行のタイムスケジュールは毎回異なるので、多くの場合は x は実際のスレッド数より少ない値を吐き出すことになるのです。

for reduction(+ : X) とします。

chapter2-3-5.cpp の実行結果を以下に示します。

```
N = 50000000
omp_get_max_threads() = 16

[No reduction (race)]
  sum = 7068296 (expected 50000000)
  error = 42931704
  time = 0.120810 sec

[Reduction]
  sum = 50000000 (expected 50000000)
  error = 0
  time = 0.002700 sec

[Atomic (reference)]
  sum = 50000000 (expected 50000000)
  error = 0
  time = 0.814223 sec
```

上の実行結果をもとに手法の違いについて述べます。まず No reduction (race) の結果では、期待される値が 50000000 であるにもかかわらず実際の値はそれより小さくなっています。これは複数のスレッドが同時に shared の変数に対して += の操作を行ったことによる競合が原因です。

次に Reduction を用いた場合、各スレッドがそれぞれ独立したローカル変数を用いて加算を行い、並列処理の終了後にそれらをまとめて合計する仕組みになっているため結果が正しく 50000000 となっています。この方法ではループの実行中に共有変数を直接更新する必要がないため、競合を回避しつつ効率的に計算を行うことができます。実行時間が非常に短くなっていることから、オーバーヘッドが小さいことが分かります。

最後に比較のため Atomic を用いた場合について述べます。Atomic は並列化こそしていますが、各スレッド1つずつ順序立てて実行するプログラムとなっています。結果は正しくなっていますが、実行時間が大幅に長くなっています。

以上より並列ループ内で総和を計算するような場合には reduction を用いることで正確性と効率の両方を満たすことができることが分かります。

Chapter 3

強束縛模型の解析

本稿では、物性理論における多くの解析の出発点として表れる強束縛模型の解析手法について述べますが、これまで述べたような対角化、フーリエ変換については一度おいておきます。のちに解説しますが、まずはバンド、フェルミ面、状態密度という基本的な解析対象について述べ、その実装方法について紹介します。

3.1 2次元正方格子の強束縛模型の構築

まず2次元正方格子上のシングルバンドの強束縛模型を実空間で構成し、その後フーリエ変換によって波数空間表示を導きます。格子定数は1とします。格子点は $\mathbf{R}_i = (x_i, y_i)$ で表され、 i 番目の格子に対しフェルミオンの生成・消滅演算子 $c_i^\dagger, c_i$ を導入します。スピン添え字は省略します。

正方格子における最近接ベクトルは

$$\boldsymbol{\delta} \in \{(\pm 1, 0), (0, \pm 1)\} \quad (3.1)$$

であり、次近接ベクトルは

$$\boldsymbol{\delta}' \in \{(\pm 1, \pm 1)\} \quad (3.2)$$

となります。最近接ホッピング t と次近接ホッピング t' を含めた実空間のハミルトニアンは

$$H = -t \sum_i \sum_{\boldsymbol{\delta}} c_i^\dagger c_{i+\boldsymbol{\delta}} - t' \sum_i \sum_{\boldsymbol{\delta}'} c_i^\dagger c_{i+\boldsymbol{\delta}'} - \mu \sum_i c_i^\dagger c_i \quad (3.3)$$

と書けます。

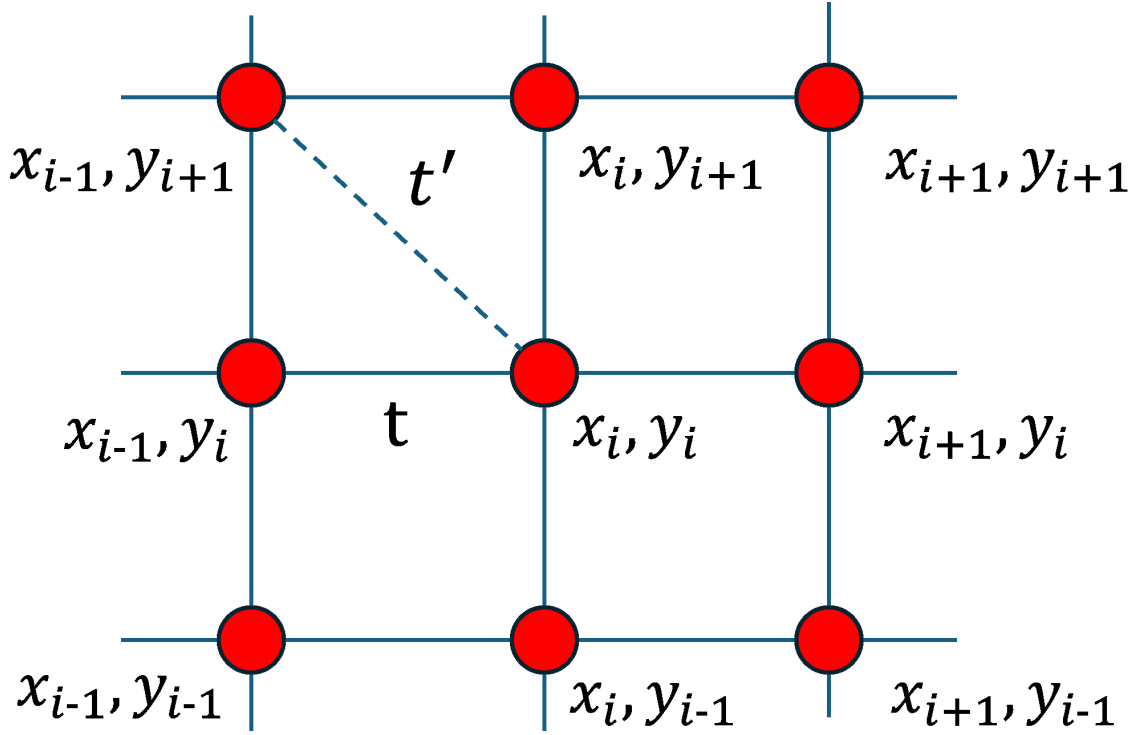


図 3.1: 正方格子の模式図。\$t\$ が最近接ホッピングを、\$t'\$ が次近接ホッピングを表す。格子定数は \$a = 1\$ としている。

ここで実空間と波数空間の演算子の関係を

$$c_i = \frac{1}{\sqrt{N}} \sum_{\mathbf{k}} e^{i\mathbf{k} \cdot \mathbf{R}_i} c_{\mathbf{k}}, \quad c_i^\dagger = \frac{1}{\sqrt{N}} \sum_{\mathbf{k}} e^{-i\mathbf{k} \cdot \mathbf{R}_i} c_{\mathbf{k}}^\dagger \quad (3.4)$$

と定義して、式 (1) のハミルトニアン の第一項

$$-t \sum_i \sum_{\delta} c_i^\dagger c_{i+\delta} \quad (3.5)$$

にフーリエ変換の表式を代入すると、

$$c_i^\dagger c_{i+\delta} = \frac{1}{N} \sum_{\mathbf{k}, \mathbf{k}'} e^{-i\mathbf{k} \cdot \mathbf{R}_i} e^{i\mathbf{k}' \cdot (\mathbf{R}_i + \delta)} c_{\mathbf{k}}^\dagger c_{\mathbf{k}'} \quad (3.6)$$

となります。第 2 項の次近接ホッピングに関する項も同様に計算することができます。

ここで格子点 \$i\$ について和を取ると、\$N\$ を格子点の数として

$$\sum_i e^{i(\mathbf{k}' - \mathbf{k}) \cdot \mathbf{R}_i} = N \delta_{\mathbf{k}, \mathbf{k}'} \quad (3.7)$$

であることを用いて

$$-t \sum_{\mathbf{k}} \left(\sum_{\delta} e^{i\mathbf{k} \cdot \delta} \right) c_{\mathbf{k}}^{\dagger} c_{\mathbf{k}} - t' \sum_{\mathbf{k}} \left(\sum_{\delta'} e^{i\mathbf{k} \cdot \delta'} \right) c_{\mathbf{k}}^{\dagger} c_{\mathbf{k}} \quad (3.8)$$

を得ます。特に最近接ホッピングのみ考慮した正方格子では

$$\sum_{\delta} e^{i\mathbf{k} \cdot \delta} = e^{ik_x} + e^{-ik_x} = 2 \cos k_x + 2 \cos k_y \quad (3.9)$$

となります。また次近接を考慮した項について和を取ると

$$\sum_{\delta'} e^{i\mathbf{k} \cdot \delta'} = e^{i(k_x+k_y)} + e^{i(k_x-k_y)} + e^{-i(k_x-k_y)} + e^{-i(k_x+k_y)} \quad (3.10)$$

$$= 2 \cos(k_x + k_y) + 2 \cos(k_x - k_y) \quad (3.11)$$

$$= 4 \cos k_x \cos k_y \quad (3.12)$$

となります。

以上をまとめると、次近接まで考慮した波数表示での2次元正方格子ハミルトニアンは

$$H = \sum_{\mathbf{k}} \varepsilon(\mathbf{k}) c_{\mathbf{k}}^{\dagger} c_{\mathbf{k}} \quad (3.13)$$

であり、分散関係は

$$\varepsilon(\mathbf{k}) = -2t(\cos k_x + \cos k_y) - 4t' \cos k_x \cos k_y - \mu \quad (3.14)$$

で与えられます。

並進対称性を仮定した場合、波数表示で既にハミルトニアンは対角的になっているので、対角化をする必要はありません。

3.2 バンド図とフェルミ面

次に、前章で導出した2次元正方格子のエネルギー分散 $\varepsilon(\mathbf{k})$ を用いて、High symmetry path に沿ったバンド図及びフェルミ面を数値計算によって実際に書いてみましょう。ここではパラメータを $t = 1.0$, $t' = -0.2$ に固定し、バンド構造を観察してみます。また化学ポテンシャル μ を変えることでフェルミ面がどのように変化するかも観察してみましょう。

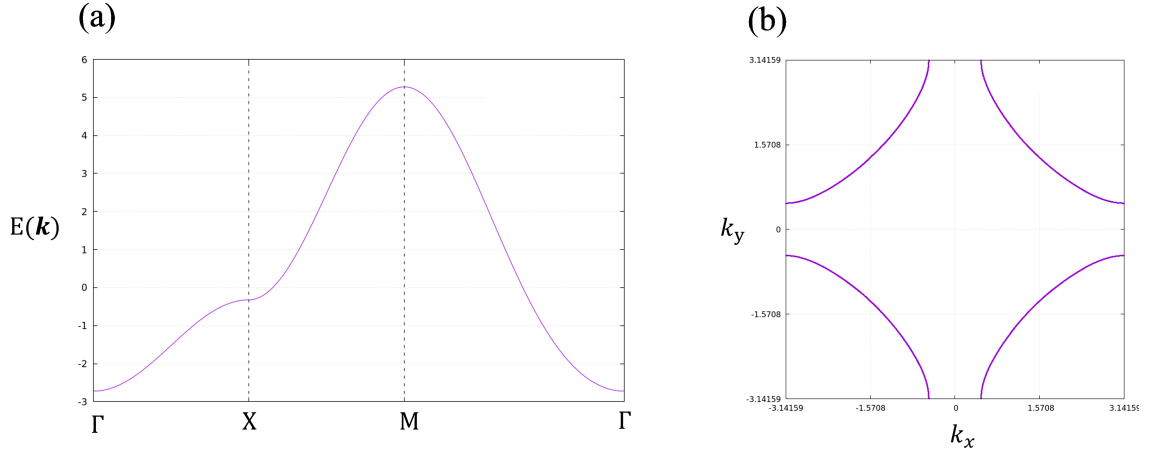


図 3.2: 正方格子の模式図。 t が最近接ホッピングを、 t' が次近接ホッピングを表す。格子定数は $a = 1$ としている。

3.2.1 バンド図の描画

2次元正方格子の 1st ブリルアンゾーンにおける代表的な高対称点は $\Gamma = (0, 0)$, $X = (\pi, 0)$, $M = (0, \pi)$ です。例えば縦軸にエネルギーをとり、平面上に波数空間を取る手法が自然に考えられますが3次元的な分散はかけても解析が難しいことのほうが多いので、高対称点を結ぶ High symmetry path に沿ってエネルギーをプロットしてバンド図とすることが多いです。

3.2.2 フェルミ面の描画

フェルミ面は $\varepsilon(k) = 0$ を満たす波数 $\mathbf{k}$ の集合からなります。金属の性質はおおかたフェルミ面近傍の電子が決めており、その形状は物理量のふるまいや発現する超伝導状態などに寄与します。そのためフェルミ面は金属の顔とも呼ばれ、出発点としてまずはフェルミ面を計算することに心血が注がれます。

フェルミ面を数値的に求める手順は、大まかには次の通りです。エネルギー分散は数値計算で離散的に求められますが、各点を線形補完し、 $\varepsilon(\mathbf{k}_i)$ と $\varepsilon(\mathbf{k}_{i+1})$ とで符号が反転する箇所を取り出します。このようにして得られたフェルミ面はあくまで点の集合ですが、メッシュサイズを十分細かくすることによって視覚的には滑らかなフェルミ面として扱うことができます。

3.2.3 化学ポテンシャル μ の自己無撞着な決定

エネルギー分散 $\varepsilon(k)$ が与えられたとき、温度 T における粒子数 n は化学ポテンシャル μ の関数として定まる。したがって、充填数 (filling) n を指定したときの化学ポテンシャル μ を計算で求めたいという場面が表れる。そこで、目標とする N に対して $n(\mu) = N$ を満たす μ を自己無撞着に求めてみましょう。

まず各 $\mathbf{k}$ の状態にある電子数は $\langle n_{\mathbf{k}} \rangle = f(\varepsilon(\mathbf{k}) - \mu)$ で与えられます。ここで

$$f(x) = \frac{1}{e^{\beta x} + 1} \quad \beta = \frac{1}{T} \quad (3.15)$$

です。スピン縮退を考えなければ filling は 0 から 1 までの値を取ります。1 サイトあたりの粒子数は、 $\frac{1}{N} \sum_{\mathbf{k}} \langle n_{\mathbf{k}} \rangle = \frac{1}{N} \sum_{\mathbf{k}} f(\varepsilon(\mathbf{k}) - \mu)$ で求まります。

さて、目標充填数 n_{target} を与えたとき、化学ポテンシャル μ は $n(\mu) - n_{target} = 0$ の根として定義されます。ここで一般に $n(\mu)$ は μ に関して単調に変化するので、二分探索によってこの解を求めることができます。

Chapter 4

付録

4.0.1 LAPACK を用いた対角化サンプルプログラム

chapter2-1.cpp

```
1 #include <iostream>
2 #include <vector>
3 #include <complex>
4 #include <stdexcept>
5 #include <fstream>
6 #include <cmath>
7 #include <limits>
8
9 using cd = std::complex<double>;
10
11 extern "C" {
12
13 void dsyevd_(
14     char* jobz, char* uplo,
15     int* n,
16     double* a, int* lda,
17     double* w,
18     double* work, int* lwork,
19     int* iwork, int* liwork,
20     int* info
21 );
```

```

22
23 void zheevd_(
24     char* jobz, char* uplo,
25     int* n,
26     cd* a, int* lda,
27     double* w,
28     cd* work, int* lwork,
29     double* rwork, int* lrwork,
30     int* iwork, int* liwork,
31     int* info
32 );
33
34 }
35
36 template <class T>
37 inline T& A(std::vector<T>& a, int i, int j, int n) {
38     return a[i + n * j];
39 }
40 template <class T>
41 inline const T& A(const std::vector<T>& a, int i, int j,
42     int n) {
43     return a[i + n * j];
44 }
45 double norm2(const std::vector<double>& x) {
46     long double s = 0.0L;
47     for (double v : x) s += (long double)v * (long double)
48         v;
49     return std::sqrt((double)s);
50 }
51 double norm2(const std::vector<cd>& x) {
52     long double s = 0.0L;
53     for (cd v : x) s += (long double)std::norm(v);
54     return std::sqrt((double)s);
55 }
56 template <class T>
57 double residual_norm2(const std::vector<T>& Aorig,
58     const std::vector<T>& V,

```

```

59         const std::vector<double>& w,
60         int n, int j) {
61     std::vector<T> r(n, T{});
62     for (int i = 0; i < n; ++i) {
63         T sum = T{};
64         for (int k = 0; k < n; ++k) {
65             sum += A(Aorig, i, k, n) * A(V, k, j, n);
66         }
67         r[i] = sum - (T)w[j] * A(V, i, j, n);
68     }
69     return norm2(r);
70 }
71
72 void diagonalize_hermitian(std::vector<cd>& a, int n, std
::vector<double>& w) {
73     w.assign(n, 0.0);
74
75     char jobz = 'V';
76     char uplo = 'U';
77     int lda = n;
78     int info = 0;
79
80     // Workspace query
81     int lwork = -1, lrwork = -1, liwork = -1;
82     cd work_query;
83     double rwork_query;
84     int iwork_query;
85
86     zheevd_(&jobz, &uplo, &n,
87            a.data(), &lda,
88            w.data(),
89            &work_query, &lwork,
90            &rwork_query, &lrwork,
91            &iwork_query, &liwork,
92            &info);
93
94     if (info != 0) {
95         throw std::runtime_error("zheevd workspace query
failed, info=" + std::to_string(info));

```

```

96     }
97
98     lwork  = (int)std::max(1.0, std::floor(work_query.real
        ())),);
99     lrwork = (int)std::max(1.0, std::floor(rwork_query));
100     liwork = std::max(1, iwork_query);
101
102     std::vector<cd> work(lwork);
103     std::vector<double> rwork(lrwork);
104     std::vector<int> iwork(liwork);
105
106     // Actual computation
107     zheevd_(&jobz, &uplo, &n,
108             a.data(), &lda,
109             w.data(),
110             work.data(), &lwork,
111             rwork.data(), &lrwork,
112             iwork.data(), &liwork,
113             &info);
114
115     if (info != 0) {
116         throw std::runtime_error("zheevd failed, info=" +
            std::to_string(info));
117     }
118 }
119
120 std::vector<cd> build_example_hermitian_3x3(int& n_out) {
121     const int n = 3;
122     n_out = n;
123     std::vector<cd> H(n * n, cd{0.0, 0.0});
124
125     // H =
126     // [ 1.0, 0.2+0.1i, 0.0
127     //    0.2-0.1i, 2.0, 0.3
128     //    0.0, 0.3, 3.0 ]
129     A(H, 0, 0, n) = 1.0;
130     A(H, 1, 1, n) = 2.0;
131     A(H, 2, 2, n) = 3.0;
132

```

```

133     cd x = cd(0.2, 0.1);
134     A(H, 0, 1, n) = x;
135     A(H, 1, 0, n) = std::conj(x);
136
137     A(H, 1, 2, n) = 0.3;
138     A(H, 2, 1, n) = 0.3;
139
140     // (0,2) = 0
141     return H;
142 }
143
144 int main() {
145     try {
146         int n = 0;
147         std::vector<cd> H = build_example_hermitian_3x3(n)
148             ;
149
150         std::vector<cd> Horig = H;
151
152         std::vector<double> w;
153         diagonalize_hermitian(H, n, w);
154
155         std::cout << "Eigenvalues (ascending):\n";
156         for (int i = 0; i < n; ++i) {
157             std::cout << "  w[" << i << "] = " << w[i] <<
158                 "\n";
159         }
160
161         std::cout << "\nEigenvectors (columns):\n";
162         for (int j = 0; j < n; ++j) {
163             std::cout << "v[" << j << "] = ( ";
164             for (int i = 0; i < n; ++i) {
165                 std::cout << A(H, i, j, n) << " ";
166             }
167             std::cout << ")\n";
168         }
169
170         double max_res = 0.0;
171         for (int j = 0; j < n; ++j) {

```

```

170         double rj = residual_norm2(Horig, H, w, n, j);
171         max_res = std::max(max_res, rj);
172     }
173     std::cout << "\nMax residual norm2: " << max_res
174         << "\n";
175
176     {
177         std::ofstream ofs("eigvals.dat");
178         ofs << "# i w[i]\n";
179         for (int i = 0; i < n; ++i) {
180             ofs << i << " " << w[i] << "\n";
181         }
182
183     {
184         std::ofstream ofs("eigvecs.dat");
185         ofs << "# columns are eigenvectors v_j; row i
186             is component i\n";
187         for (int i = 0; i < n; ++i) {
188             for (int j = 0; j < n; ++j) {
189                 cd v = A(H, i, j, n);
190                 ofs << v.real() << " " << v.imag();
191                 if (j + 1 < n) ofs << " ";
192             }
193             ofs << "\n";
194         }
195
196         return 0;
197     } catch (const std::exception& e) {
198         std::cerr << "ERROR: " << e.what() << "\n";
199         return 1;
200     }
201 }

```

4.0.2 FFTW を用いた高速フーリエ変換サンプルプログラム

chapter2-2.cpp

```
1  #include <fftw3.h>
2
3  #include <cmath>
4  #include <cstdlib>
5  #include <fstream>
6  #include <iostream>
7  #include <limits>
8
9  static inline int idx(int ix, int iy, int Ny) { return ix
    * Ny + iy; }
10
11 int main(int argc, char** argv) {
12     int Nx = 128;
13     int Ny = 128;
14     double gamma = 6.0;
15
16     if (argc >= 3) {
17         Nx = std::atoi(argv[1]);
18         Ny = std::atoi(argv[2]);
19     }
20     if (argc >= 4) {
21         gamma = std::atof(argv[3]);
22     }
23     if (Nx <= 0 || Ny <= 0) {
24         std::cerr << "Nx, Ny must be positive.\n";
25         return 1;
26     }
27     if (gamma <= 0.0) {
28         std::cerr << "gamma must be positive.\n";
29         return 1;
30     }
31
32     fftw_complex* in = (fftw_complex*)fftw_malloc(sizeof(
        fftw_complex) * Nx * Ny);
```



```

33     fftw_complex* out = (fftw_complex*)fftw_malloc(sizeof(
        fftw_complex) * Nx * Ny);
34     if (!in || !out) {
35         std::cerr << "fftw_malloc failed.\n";
36         fftw_free(in);
37         fftw_free(out);
38         return 1;
39     }
40
41     for (int ix = 0; ix < Nx; ++ix) {
42         const double x = ix - Nx / 2.0;    // 座標
43         : [-Nx/2, ..., Nx/2)
44         for (int iy = 0; iy < Ny; ++iy) {
45             const double y = iy - Ny / 2.0;
46
47             const double val = (gamma * gamma) / (x * x +
                y * y + gamma * gamma);
48
49             // シフトして (0,0) に中心を置く
50             const int ix0 = (ix + Nx / 2) % Nx;
51             const int iy0 = (iy + Ny / 2) % Ny;
52
53             in[idx(ix0, iy0, Ny)][0] = val;    // 実部
54             in[idx(ix0, iy0, Ny)][1] = 0.0;    // 虚部
55         }
56     }
57
58     // FFTW plan の作成及び実行
59     fftw_plan plan = fftw_plan_dft_2d(Nx, Ny, in, out,
        FFTW_FORWARD, FFTW_ESTIMATE);
60     if (!plan) {
61         std::cerr << "fftw_plan_dft_2d failed\n";
62         fftw_free(in);
63         fftw_free(out);
64         return 1;
65     }
66     fftw_execute(plan);
67
68     {

```

```

68         std::ofstream ofs("lorentzian_real.dat");
69         ofs << "# x y f(x,y)\n";
70         for (int x = -Nx / 2; x < Nx / 2; ++x) {
71             // x に対応する元の ix
72             const int ix = x + Nx / 2;
73             for (int y = -Ny / 2; y < Ny / 2; ++y) {
74                 const int iy = y + Ny / 2;
75
76                 // in[] は前向きにシフトして格納したので、逆シフトで
77                 // 読む
78                 const int ix0 = (ix + Nx / 2) % Nx;
79                 const int iy0 = (iy + Ny / 2) % Ny;
80
81                 ofs << x << " " << y << " " << in[idx(ix0,
82                     iy0, Ny)][0] << "\n";
83             }
84             ofs << "\n";
85         }
86     }
87
88     //
89
90     // 3) 空間 k: の結果と、連続変換の厳密解を「同じ格子点」で比較して出力 FFT
91
92     // の周波数 FFTW:
93     //  $k_x = \pi 2 * m_x / N_x$ ,  $m_x = -N_x/2..N_x$  (表示用)  $/2-1$ 
94     //  $k_y = \pi 2 * m_y / N_y$ 
95     //
96     // 連続変換の厳密解 (無限平面) :
97     //  $f(r) = \text{gamma}^2 / (r^2 + \text{gamma}^2)$ 
98     //  $F(k) = \pi 2 * \text{gamma}^2 * K0(\text{gamma} * |k|)$ 
99     //
100    // 注意:  $k=0$  は対数発散 (無限大) なので比較不能。
101    // ここでは NaN を出しておく (で弾ける) gnuplot
102    //

```

```

100     {
101         std::ofstream ofs("lorentzian_k_fft_exact.dat");
102         ofs << "# mx my kx ky |F_fft| Re(F_fft) Im(F_fft)
           F_exact\n";
103
104         const double pi = M_PI;
105
106         for (int sx = 0; sx < Nx; ++sx) {
107             const int mx = sx - Nx / 2;           //
              表示用 ()
              fftshifted
108             const int ix = (sx + Nx / 2) % Nx;     //
              実際
              の FFTW index
109             const double kx = 2.0 * pi * mx / Nx;  //
              物理波数 (dx)
              =1
110
111             for (int sy = 0; sy < Ny; ++sy) {
112                 const int my = sy - Ny / 2;
113                 const int iy = (sy + Ny / 2) % Ny;
114                 const double ky = 2.0 * pi * my / Ny;
115
116                 const double re = out[idx(ix, iy, Ny)][0];
117                 const double im = out[idx(ix, iy, Ny)][1];
118                 const double mag = std::sqrt(re * re + im
                    * im);
119
120                 const double kabs = std::sqrt(kx * kx + ky
                    * ky);
121
122                 double exact;
123                 if (kabs < 1e-14) {
124                     exact = std::numeric_limits<double>::
                        quiet_NaN(); // 発散点
                        は NaN
125                 } else {
126                     exact = 2.0 * pi * gamma * gamma * std
                        ::cyl_bessel_k(0, gamma * kabs);

```

```

127         }
128
129         ofs << mx << " " << my << " "
130         << kx << " " << ky << " "
131         << mag << " " << re << " " << im << "
132         << exact << "\n";
133     }
134     ofs << "\n";
135 }
136 }
137
138 fftw_destroy_plan(plan);
139 fftw_free(in);
140 fftw_free(out);
141
142 std::cout << "Done.\n"
143 << "Outputs:\n"
144 << "  - lorentzian_real.dat\n"
145 << "  - lorentzian_k_fft_exact.dat\n";
146 return 0;
147 }

```

4.0.3 OpenMP による並列計算サンプルプログラム

chapter2-3.cpp

```

1  #include <iostream>
2  #include <omp.h>
3
4  int main() {
5  #pragma omp parallel
6      {
7          int tid = omp_get_thread_num();
8          int nthreads = omp_get_num_threads();
9          printf("thread %d / %d\n", tid, nthreads);
10     }

```

```
11     return 0;
12 }
```

chapter2-3-2.cpp

```
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     #pragma omp parallel for
6     for (int i = 0; i < 8; ++i) {
7         printf("i = %d, thread = %d\n", i,
8             omp_get_thread_num());
9     }
10 }
```

chapter2-3-3.cpp

```
1 #include <iostream>
2 #include <omp.h>
3
4 int main() {
5     int x = 0;
6
7     #pragma omp parallel
8     {
9         x += 1;
10    }
11
12    std::cout << x << std::endl;
13 }
```

chapter2-3-4.cpp

```
1 #include <omp.h>
2 #include <cstdio>
3 #include <cstdint>
4 #include <cstdlib>
5 #include <iostream>
```

```

6 #include <iomanip>
7
8 int main(int argc, char** argv) {
9     // 反復回数 (十分大きい方が差が出やすい)
10    // 例: ./reduction_demo 50000000
11    long long N = 50'000'000LL;
12    if (argc >= 2) {
13        N = std::atoll(argv[1]);
14        if (N <= 0) {
15            std::cerr << "N must be positive.\n";
16            return 1;
17        }
18    }
19
20    // のスレッド数 (環境変数 OpenMP OMP_NUM_THREADS が優先されるの
    // が普通)
21    int max_threads = omp_get_max_threads();
22
23    std::cout << "N = " << N << "\n";
24    std::cout << "omp_get_max_threads() = " << max_threads
    << "\n";
25
26    // ---- (A) reduction なし : データ競合が起きる ----
27    long long sum_race = 0;
28    double t0 = omp_get_wtime();
29
30    #pragma omp parallel for
31    for (long long i = 0; i < N; ++i) {
32        // 競合しやすい典型 : 共有変数に +=
33        sum_race += 1;
34    }
35
36    double t1 = omp_get_wtime();
37
38    // ---- (B) reduction あり : 正しい ----
39    long long sum_reduction = 0;
40    double t2 = omp_get_wtime();
41
42    #pragma omp parallel for reduction(+:sum_reduction)

```

```

43     for (long long i = 0; i < N; ++i) {
44         sum_reduction += 1;
45     }
46
47     double t3 = omp_get_wtime();
48
49     // ---- (C) 参考: (正しいが遅くなりがち) atomic ----
50     long long sum_atomic = 0;
51     double t4 = omp_get_wtime();
52
53 #pragma omp parallel for
54     for (long long i = 0; i < N; ++i) {
55 #pragma omp atomic
56         sum_atomic += 1;
57     }
58
59     double t5 = omp_get_wtime();
60
61     // 期待値 (逐次なら必ず) N
62     const long long expected = N;
63
64     std::cout << std::fixed << std::setprecision(6);
65
66     std::cout << "\n[No reduction (race)]\n";
67     std::cout << "    sum = " << sum_race << " (expected "
68         << expected << ")\n";
69     std::cout << "    error = " << (expected - sum_race) <<
70         "\n";
71     std::cout << "    time = " << (t1 - t0) << " sec\n";
72
73     std::cout << "\n[Reduction]\n";
74     std::cout << "    sum = " << sum_reduction << " (
75         expected " << expected << ")\n";
76     std::cout << "    error = " << (expected - sum_reduction
77         ) << "\n";
78     std::cout << "    time = " << (t3 - t2) << " sec\n";
79
80     std::cout << "\n[Atomic (reference)]\n";
81     std::cout << "    sum = " << sum_atomic << " (expected "

```

```

        << expected << ")\n";
78     std::cout << "    error = " << (expected - sum_atomic)
        << "\n";
79     std::cout << "    time = " << (t5 - t4) << " sec\n";
80
81     std::cout << "\nTip: run multiple times; the 'race'
        result often changes.\n";
82     return 0;
83 }

```

chapter3.cpp

```

1  #include <cmath>
2  #include <cstdlib>
3  #include <fstream>
4  #include <iomanip>
5  #include <iostream>
6  #include <limits>
7  #include <sstream>
8  #include <stdexcept>
9  #include <string>
10 #include <vector>
11
12 #include <omp.h>
13
14 static constexpr double PI =
    3.1415926535897932384626433832795;
15
16 struct KPoint { double kx, ky; };
17
18 // ----- model dispersion (without mu) -----
19 static inline double eps_k(double kx, double ky, double t,
    double tp){
20     return -2.0*t*(std::cos(kx) + std::cos(ky))
21         -4.0*tp*(std::cos(kx)*std::cos(ky));
22 }
23
24 // ----- stable fermi function  $f(x)=1/(e^x+1)$ 
    -----

```



```

25 static inline double fermi(double x){
26     if (x > 50.0) return 0.0;
27     if (x < -50.0) return 1.0;
28     return 1.0 / (std::exp(x) + 1.0);
29 }
30
31 // centered k in (-pi, pi)
32 static inline double k_centered(int i, int Nk){
33     return 2.0*PI*( (i + 0.5)/double(Nk) - 0.5 );
34 }
35
36 // ----- estimate band edges (parallel) -----
37 static void estimate_band_edges(double t, double tp, int
    Nk_edge, double &emin, double &emax){
38     double loc_min = +1e300;
39     double loc_max = -1e300;
40
41 #pragma omp parallel
42 {
43     double tmin = +1e300;
44     double tmax = -1e300;
45
46 #pragma omp for collapse(2) nowait
47     for(int iy=0; iy<Nk_edge; ++iy){
48         for(int ix=0; ix<Nk_edge; ++ix){
49             const double ky = k_centered(iy, Nk_edge);
50             const double kx = k_centered(ix, Nk_edge);
51             const double e = eps_k(kx, ky, t, tp);
52             if(e < tmin) tmin = e;
53             if(e > tmax) tmax = e;
54         }
55     }
56
57 #pragma omp critical
58 {
59     if(tmin < loc_min) loc_min = tmin;
60     if(tmax > loc_max) loc_max = tmax;
61 }
62 }

```

```

63
64     emin = loc_min;
65     emax = loc_max;
66 }
67
68 // ----- filling n(mu) per site (spinful g=2)
69 -----
69 static double filling_n(double mu, double t, double tp,
70     double T, int Nk, int g=2){
71     const double beta = (T > 0.0) ? (1.0/T) : std::
        numeric_limits<double>::infinity();
71     double sum = 0.0;
72
73 #pragma omp parallel for collapse(2) reduction(+:sum)
74     for(int iy=0; iy<Nk; ++iy){
75         for(int ix=0; ix<Nk; ++ix){
76             const double ky = k_centered(iy, Nk);
77             const double kx = k_centered(ix, Nk);
78             const double e = eps_k(kx, ky, t, tp);
79             if(T > 0.0) sum += fermi(beta*(e - mu));
80             else      sum += (e < mu) ? 1.0 : 0.0;
81         }
82     }
83
84     return double(g) * sum / (double(Nk)*double(Nk));
85 }
86
87 // ----- solve mu for target filling by bisection
88 -----
88 static double solve_mu_bisect(double n_target, double t,
89     double tp, double T, int Nk,
90     int g=2, double tol_n=1e-10,
91     int max_iter=200){
92     if(n_target < 0.0 || n_target > double(g)){
93         throw std::runtime_error("n_target out of range.
94             For spinful g=2: n in [0,2].");
95     }
96
97     double emin, emax;

```

```

95     estimate_band_edges(t, tp, 700, emin, emax);
96
97     const double pad = (T > 0.0) ? (20.0*T) : 5.0;
98     double mu_lo = emin - pad;
99     double mu_hi = emax + pad;
100
101     double n_lo = filling_n(mu_lo, t, tp, T, Nk, g);
102     double n_hi = filling_n(mu_hi, t, tp, T, Nk, g);
103
104     int widen = 0;
105     while((n_lo > n_target || n_hi < n_target) && widen <
106           25){
107         mu_lo -= 2.0*pad;
108         mu_hi += 2.0*pad;
109         n_lo = filling_n(mu_lo, t, tp, T, Nk, g);
110         n_hi = filling_n(mu_hi, t, tp, T, Nk, g);
111         widen++;
112     }
113     if(n_lo > n_target || n_hi < n_target){
114         throw std::runtime_error("Failed to bracket mu.
115                                   Try larger Nk or T>0.");
116     }
117
118     double mu_mid = 0.0;
119     for(int it=0; it<max_iter; ++it){
120         mu_mid = 0.5*(mu_lo + mu_hi);
121         const double n_mid = filling_n(mu_mid, t, tp, T,
122                                         Nk, g);
123         const double err = n_mid - n_target;
124         if(std::abs(err) < tol_n) return mu_mid;
125
126         if(n_mid < n_target) mu_lo = mu_mid;
127         else mu_hi = mu_mid;
128     }
129     return mu_mid;
130 }
131
132 // ----- high-symmetry path  $\Gamma \rightarrow X \rightarrow M \Gamma \rightarrow$  -----
133 static std::vector<KPoint> make_path(int nseg){

```

```

131     std::vector<KPoint> pts;
132     pts.reserve(3*nseg + 1);
133     auto add_seg = [&](KPoint a, KPoint b){
134         for(int i=0;i<nseg;i++){
135             const double s = double(i)/double(nseg);
136             pts.push_back({a.kx*(1.0-s)+b.kx*s, a.ky*(1.0-
                s)+b.ky*s});
137         }
138     };
139     const KPoint G{0.0,0.0}, X{PI,0.0}, M{PI,PI};
140     add_seg(G,X);
141     add_seg(X,M);
142     add_seg(M,G);
143     pts.push_back(G);
144     return pts;
145 }
146
147 static inline double lerp_zero(double a, double b){
148     const double denom = (b-a);
149     if(std::abs(denom) < 1e-14) return 0.5;
150     return -a/denom;
151 }
152
153 static std::string tag_num(double x){
154     std::ostringstream ss;
155     ss<<std::fixed<<std::setprecision(4)<<x;
156     std::string s=ss.str();
157     for(char& c: s){
158         if(c=='.') c='p';
159         if(c=='-') c='m';
160     }
161     return s;
162 }
163
164 // ----- DOS with Gaussian broadening (parallel,
165 // thread-local accumulate) -----
166 static void compute_dos_gaussian(double t, double tp, int
    Nk, int g,
    int nE, double eta,

```

```

167                                     const std::string&
                                       dos_dat){
168     double emin, emax;
169     estimate_band_edges(t, tp, 900, emin, emax);
170
171     const double Epad = 6.0*eta;
172     const double E0 = emin - Epad;
173     const double E1 = emax + Epad;
174     const double dE = (E1 - E0) / double(nE-1);
175
176     std::vector<double> dos(nE, 0.0);
177     const double norm_gauss = 1.0 / (std::sqrt(2.0*PI) *
                                       eta);
178
179     const int nthreads = omp_get_max_threads();
180     std::vector<std::vector<double>> dos_local(nthreads,
                                       std::vector<double>(nE, 0.0));
181
182     #pragma omp parallel
183     {
184         const int tid = omp_get_thread_num();
185         auto &buf = dos_local[tid];
186
187         #pragma omp for collapse(2) nowait
188         for(int iy=0; iy<Nk; ++iy){
189             for(int ix=0; ix<Nk; ++ix){
190                 const double ky = k_centered(iy, Nk);
191                 const double kx = k_centered(ix, Nk);
192                 const double e = eps_k(kx, ky, t, tp);
193
194                 // accumulate contribution of this k-point
195                 to all energies
196                 for(int iE=0; iE<nE; ++iE){
197                     const double E = E0 + dE*double(iE);
198                     const double x = (E - e)/eta;
199                     buf[iE] += norm_gauss * std::exp(-0.5*
200                                     x*x);
199             }
200     }

```

```

201     }
202 }
203
204 // reduce buffers
205 for(int tid=0; tid<nthreads; ++tid){
206     for(int iE=0; iE<nE; ++iE){
207         dos[iE] += dos_local[tid][iE];
208     }
209 }
210
211 const double pref = double(g) / (double(Nk)*double(Nk)
    );
212 std::ofstream ofs(dos_dat);
213 ofs << std::setprecision(16);
214 ofs << "# E   DOS(E)   (Gaussian eta="<<eta<<")\n";
215 for(int iE=0; iE<nE; ++iE){
216     const double E = E0 + dE*double(iE);
217     ofs << E << " " << pref * dos[iE] << "\n";
218 }
219 }
220
221 int main(int argc, char** argv){
222     // ---- defaults ----
223     double t   = 1.0;
224     double tp  = -0.2;
225
226     double n_target = 1.0; // spinful: [0,2]
227     double T = 0.02;
228     int Nk_mu = 900;
229     int Nk_fs = 700;
230     int nseg  = 260;
231
232     int nE = 1200;
233     double eta = 0.02;
234
235     // Usage:
236     // ./tbpipe [n_target] [T] [Nk_mu] [Nk_fs] [eta] [t] [
    tp]
237     if(argc >= 2) n_target = std::atof(argv[1]);

```

```

238     if(argc >= 3) T          = std::atof(argv[2]);
239     if(argc >= 4) Nk_mu      = std::atoi(argv[3]);
240     if(argc >= 5) Nk_fs      = std::atoi(argv[4]);
241     if(argc >= 6) eta        = std::atof(argv[5]);
242     if(argc >= 7) t          = std::atof(argv[6]);
243     if(argc >= 8) tp         = std::atof(argv[7]);
244
245     if(Nk_mu < 80) Nk_mu = 80;
246     if(Nk_fs < 80) Nk_fs = 80;
247     if(nseg < 50) nseg = 50;
248     if(nE < 200) nE = 200;
249     if(eta <= 0.0) throw std::runtime_error("eta must be >
        0.");
250     if(T < 0.0) throw std::runtime_error("T must be >=
        0.");
251
252     const int g = 2;
253     const double tol_n = 1e-10;
254
255     std::cout << std::setprecision(16);
256     std::cout << "OpenMP threads = " <<
        omp_get_max_threads() << "\n";
257     std::cout << "params: t="<<t<<" tp="<<tp<<" n_target=
        "<<n_target
258             << " T="<<T<<" Nk_mu="<<Nk_mu<<" Nk_fs="<<
        Nk_fs
259             << " eta="<<eta<<"\n";
260
261     // 1) solve mu
262     const double mu = solve_mu_bisect(n_target, t, tp, T,
        Nk_mu, g, tol_n, 200);
263     const double n_check = filling_n(mu, t, tp, T, Nk_mu,
        g);
264
265     std::cout << "mu_solved = " << mu << "\n";
266     std::cout << "n(mu_solved) = " << n_check << " err="
        << (n_check - n_target) << "\n";
267
268     // tags

```

```

269     const std::string tag = "n"+tag_num(n_target)+"_T"+
        tag_num(T)+"_t"+tag_num(t)+"_tp"+tag_num(tp);
270     const std::string mu_tag = "mu"+tag_num(mu);
271
272     const std::string band_dat = "band_"+tag+"_"+mu_tag+".
        dat";
273     const std::string fs_dat = "fs_"+tag+"_"+mu_tag+".
        dat";
274     const std::string dos_dat = "dos_"+tag+"_eta"+tag_num
        (eta)+".dat";
275
276     const std::string band_plt = "plot_band_"+tag+"_"+
        mu_tag+".plt";
277     const std::string fs_plt = "plot_fs_"+tag+"_"+mu_tag
        +".plt";
278     const std::string dos_plt = "plot_dos_"+tag+"_eta"+
        tag_num(eta)+".plt";
279
280     // 2) band
281     {
282         std::ofstream ofs(band_dat);
283         ofs<<std::setprecision(16);
284         ofs<<"# s E(k)=eps-mu kx ky\n";
285
286         auto path = make_path(nseg);
287         double sdist = 0.0;
288         for(size_t i=0;i<path.size();++i){
289             if(i>0){
290                 const double dx = path[i].kx - path[i-1].
                    kx;
291                 const double dy = path[i].ky - path[i-1].
                    ky;
292                 sdist += std::sqrt(dx*dx + dy*dy);
293             }
294             const double e = eps_k(path[i].kx, path[i].ky,
                t, tp) - mu;
295             ofs<<sdist<<" "<<e<<" "<<path[i].kx<<" "<<path
                [i].ky<<"\n";
296         }

```



```

297     }
298     // gnuplot for band
299     {
300         const double dGX = PI;
301         const double dXM = PI;
302         const double dMG = std::sqrt(2.0)*PI;
303         const double sG0 = 0.0;
304         const double sX  = sG0 + dGX;
305         const double sM  = sX + dXM;
306         const double sG1 = sM + dMG;
307
308         std::ofstream ofs(band_plt);
309         ofs<<"set terminal pngcairo size 1000,700\n";
310         ofs<<"set output 'band_ "<<tag<<"_"<<mu_tag<<".png
            '\n";
311         ofs<<"set xlabel 'k-path'\n";
312         ofs<<"set ylabel 'E(k) = eps(k) - mu'\n";
313         ofs<<"set grid\n";
314         ofs<<"set title 'Band (OpenMP): t-t\\047 square, "
            <<tag<< ", "<<mu_tag<<"'\n";
315         ofs<<"set arrow from "<<sX<< ", graph 0 to "<<sX<< "
            , graph 1 nohead dt 2\n";
316         ofs<<"set arrow from "<<sM<< ", graph 0 to "<<sM<< "
            , graph 1 nohead dt 2\n";
317         ofs<<"set xtics ('{/Symbol G}' "<<sG0<< ", 'X' "<<
            sX<< ", 'M' "<<sM<< ", '{/Symbol G}' "<<sG1<< ") \n
            ";
318         ofs<<"plot '"<<band_dat<<"' using 1:2 with lines
            title 'E(k)'\n";
319     }
320
321     // 3) FS extraction (parallel over cells; thread-local
vectors)
322     {
323         const double dk = 2.0*PI/double(Nk_fs);
324         auto idx = [&](int ix, int iy){ return ix + (Nk_fs
            +1)*iy; };
325         std::vector<double> E((Nk_fs+1)*(Nk_fs+1), 0.0);
326         const double eps0 = 1e-3;

```

```

327
328         // build grid  $E(k)=\epsilon-\mu$ 
329 #pragma omp parallel for collapse(2)
330     for(int iy=0; iy<=Nk_fs; ++iy){
331         for(int ix=0; ix<=Nk_fs; ++ix){
332             const double ky = -PI + dk*double(iy);
333             const double kx = -PI + dk*double(ix);
334             E[idx(ix,iy)] = eps_k(kx,ky,t,tp) - mu;
335         }
336     }
337
338     // per-thread buffers for FS points
339     const int nthreads = omp_get_max_threads();
340     std::vector<std::vector<KPoint>> fs_local(nthreads
341 );
342 #pragma omp parallel
343 {
344     const int tid = omp_get_thread_num();
345     auto &buf = fs_local[tid];
346     buf.reserve(size_t(Nk_fs)*size_t(Nk_fs)/
347 nthreads);
348
349     auto add_edge = [&](double eA, double eB,
350 double kxA, double kyA, double kxB, double
351 kyB){
352         if(std::abs(eA) < eps0){ buf.push_back({
353 kxA,kyA}); return; }
354         if(std::abs(eB) < eps0){ buf.push_back({
355 kxB,kyB}); return; }
356         if(eA*eB < 0.0){
357             const double s = lerp_zero(eA,eB);
358             buf.push_back({kxA + s*(kxB-kxA), kyA
359 + s*(kyB-kyA)});
360         }
361     };
362
363 #pragma omp for collapse(2) nowait
364     for(int iy=0; iy<Nk_fs; ++iy){

```

```

359         for(int ix=0; ix<Nk_fs; ++ix){
360             const double ky0 = -PI + dk*double(iy)
361             ;
362             const double ky1 = ky0 + dk;
363             const double kx0 = -PI + dk*double(ix)
364             ;
365             const double kx1 = kx0 + dk;
366
367             const double e00 = E[idx(ix, iy )];
368             const double e10 = E[idx(ix+1,iy )];
369             const double e01 = E[idx(ix, iy+1)];
370             const double e11 = E[idx(ix+1,iy+1)];
371
372             add_edge(e00,e10,kx0,ky0,kx1,ky0);
373             add_edge(e01,e11,kx0,ky1,kx1,ky1);
374             add_edge(e00,e01,kx0,ky0,kx0,ky1);
375             add_edge(e10,e11,kx1,ky0,kx1,ky1);
376         }
377     }
378
379     // merge
380     std::vector<KPoint> fs;
381     size_t total = 0;
382     for(auto &v: fs_local) total += v.size();
383     fs.reserve(total);
384     for(auto &v: fs_local){
385         fs.insert(fs.end(), v.begin(), v.end());
386     }
387
388     std::ofstream ofs(fs_dat);
389     ofs<<std::setprecision(16);
390     ofs<<"# kx ky (FS points: E(k)=0)\n";
391     for(const auto& p: fs) ofs<<p.kx<<" "<<p.ky<<"\n";
392 }
393
394 // gnuplot for FS
395 {
396     std::ofstream ofs(fs_plt);

```

```

396         ofs<<"set terminal pngcairo size 850,850\n";
397         ofs<<"set output 'fs_"<<tag<<"_"<<mu_tag<<".png'\n
           ";
398         ofs<<"set size ratio -1\n";
399         ofs<<"set xrange [-pi:pi]\n";
400         ofs<<"set yrange [-pi:pi]\n";
401         ofs<<"set xlabel 'k_x'\n";
402         ofs<<"set ylabel 'k_y'\n";
403         ofs<<"set grid\n";
404         ofs<<"set title 'FS (OpenMP): "<<tag<< ", "<<mu_tag
           <<"'\n";
405         ofs<<"pi=3.141592653589793\n";
406         ofs<<"set xtics (-pi, -pi/2, 0, pi/2, pi)\n";
407         ofs<<"set ytics (-pi, -pi/2, 0, pi/2, pi)\n";
408         ofs<<"plot '"<<fs_dat<<"' using 1:2 with points pt
           7 ps 0.25 title 'FS points'\n";
409     }
410
411     // 4) DOS
412     compute_dos_gaussian(t, tp, Nk_mu, g, nE, eta, dos_dat
413         );
414
415     // gnuplot for DOS
416     {
417         std::ofstream ofs(dos_plt);
418         ofs<<"set terminal pngcairo size 1000,700\n";
419         ofs<<"set output 'dos_"<<tag<<"_eta"<<tag_num(eta)
           <<".png'\n";
420         ofs<<"set xlabel 'E'\n";
421         ofs<<"set ylabel 'DOS(E) (per site, spin included
           )'\n";
422         ofs<<"set grid\n";
423         ofs<<"set title 'DOS (OpenMP): "<<tag<< ", eta="<<
           eta<<"'\n";
424         ofs<<"plot '"<<dos_dat<<"' using 1:2 with lines
           title 'DOS'\n";
425     }
426
427     std::cout << "\n[done] wrote:\n"

```

```

427         << "    mu = " << mu << "\n"
428         << "    " << band_dat << "\n"
429         << "    " << fs_dat    << "\n"
430         << "    " << dos_dat   << "\n"
431         << "    " << band_plt  << "\n"
432         << "    " << fs_plt   << "\n"
433         << "    " << dos_plt   << "\n\n";
434
435     std::cout << "Run gnuplot:\n"
436         << "    gnuplot " << band_plt << "\n"
437         << "    gnuplot " << fs_plt    << "\n"
438         << "    gnuplot " << dos_plt   << "\n";
439
440     return 0;
441 }

```

Chapter 4

関連図書